

6장 연결 리스트1

□ 자기 참조 구조체 (self-referential structure)

- 구조체를 정의할 때 멤버중에 자기 자신을 가리키는 포인터가 존재하는 구조체

```
typedef struct ListNode {  
    char data[10];  
    struct ListNode* link; // 자신과 같은 구조체를 가리킬 포인터  
} Node;
```

Node* link; 로 대신 쓸수 있을까?

리스트(List)란?

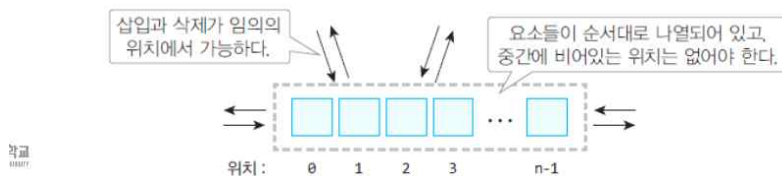
일상생활에서의 리스트

오늘 해야 할 일, 버킷 리스트, 요일들(월요일, 화요일, ... , 일요일)

리스트 : 순서를 가진 항목들의 모임 <-> 집합은 항목간의 순서 개념이 없음

연산 : 삽입, 삭제, 탐색

□ 구조



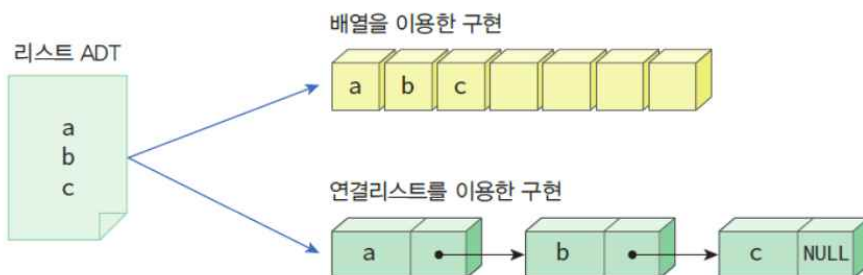
리스트의 추상 자료형(ADT)

데이터: 같은 유형의 요소들의 순서 있는 모임

연산

- insert(pos, e): pos 위치에 새로운 요소 e를 삽입한다.
- delete(pos): pos 위치에 있는 요소를 꺼내고(삭제) 반환한다.
- is_empty(): 리스트가 비어 있는지를 검사한다.
- is_full(): 리스트가 가득 차 있는지를 검사한다.
- get_entry(pos): pos 위치에 있는 요소를 반환한다.

리스트는 배열 또는 연결리스트로 구현할 수 있다.



배열을 이용한 리스트

size라는 포인터가 왔다갔다하면서 리스트의 크기를 조절.

Empty는 size==0, Full은 size==MAX_SIZE로 공백, 포화상태 점검

insert(pos, e) : pos위치에 새로운 요소 e 삽입. 원래 있던것들은 뒤로 shift. O(n)

delete(pos) : pos위치의 요소를 꺼내서 반환. 뒤에 있는 것은 앞으로.

O(n)

get_entry(pos) : pos위치의 요소를 참조. 삭제하지 않음.

```
#define MAX_SIZE 100 // 리스트의 최대크기
typedef int element; // 항목의 정의

typedef struct {
    element data[MAX_SIZE]; // 배열 정의
    int size; // 현재 리스트
} ArrayListType;

// 오류 처리 함수
void error(char *message)
{
    fprintf(stderr, "%s\n", message);
    exit(1);
}

// 리스트 초기화 함수
void init(ArrayListType *L)
{
    L->size = 0;
}
```

```
int is_empty(ArrayListType *L)
{
    return L->size == 0; //비어 있으면 1, 그렇지 않으면 0을 반환
}

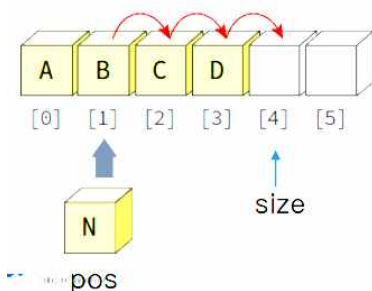
int is_full(ArrayListType *L)
{
    return L->size == MAX_SIZE; //가득 차있으면 1, 그렇지 않으면 0을 반환
}

element get_entry(ArrayListType *L, int pos)
{
    if (pos < 0 || pos >= L->size) //유효하지 않은 인덱스 위치
        error("Invalid Position Error");
    return L->data[pos];
}

// 리스트 출력
void print_list(ArrayListType *L)
{
    int i;
    for (i = 0; i < L->size; i++)
        printf("%d->", L->data[i]);
    printf("\n");
}
```

```
void insert(ArrayListType *L, int pos, element item)
{
    if (is_full(L))
        error("Overflow Error");
    if (pos < 0 || (pos > L->size))
        error("Invalid Position Error");

    for (int i = (L->size - 1); i >= pos; i--) {
        L->data[i + 1] = L->data[i];
    }
    L->data[pos] = item;
    L->size++;
}
```



```
//같은 내용 다른 표현 insert() 함수
void insert(ArrayListType *L, int pos, element item)
{
    if (!is_full(L) && (pos >= 0) && (pos <= L->size)) {
        for (int i = (L->size - 1); i >= pos; i--)
            L->data[i + 1] = L->data[i];
        L->data[pos] = item;
        L->size++;
    }
}
```

리스트에서의 새로운 연산자

append(): 리스트 맨 뒤에 요소를 추가

pop(): 리스트 맨 뒤의 요소를 꺼내서 반환

replace(): 해당 위치 요소를 다른 요소로 수정(대체)

find(): 해당 요소가 리스트에 존재하면 해당 위치(인덱스)를 반환, 없을 경우 -1을 반환

>> 리스트는 사용이 자유로워서 여러가지 연산자를 추가 가능

//리스트 맨끝에 요소를 추가

```
void append(ArrayListType *L, element item)
{
    insert(&L, L->size, item);
}
```

O(1)

```
//리스트 맨 끝 요소를 꺼내서 (삭제 후) 반환
element pop(ArrayListType *L)
```

$O(1)$

```
{
    return delete(&L, L->size-1);
}
```

```
//pos 위치의 데이터를 e로 수정
```

```
void replace(ArrayListType *L, int pos, element item)
```

$O(1)$

```
{
    if( (pos < 0) || (pos >= L->size) )
        error("Invalid Position Error");
    L->data[pos] = item;
}
```

```
//item을 찾으면 인덱스, 없으면 -1을 반환
```

```
int find(ArrayListType *L, element item)
```

$O(n)$

```
{
    for(int i = 0; i < L->size; i++)
        if(L->data[i] == item)
            return i;
    return -1;
}
```

```
int main(void)
```

```
{
    // ArrayListType를 정적으로 생성하고 ArrayListType를 가리키는 포인터를
    //함수의 매개변수로 전달한다.
```

```
ArrayListType list;
```

```
init(&list);
```

```
insert(&list, 0, 10);
```

```
insert(&list, 0, 20);
```

```
insert(&list, 0, 30);
```

```
append(&list, 40);
```

```
delete(&list, 0);
```

```
return 0;
```

```
print_list(&list);
```

```
print_list(&list);
```

```
print_list(&list);
```

```
print_list(&list);
```

```
print_list(&list);
```

```
// 0번째 위치에 10 추가
```

```
// 0번째 위치에 20 추가
```

```
// 0번째 위치에 30 추가
```

```
// 맨 끝에 40 추가
```

```
// 0번째 항목 삭제
```

```
}
```

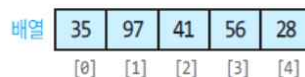
출력 : 10->

20->10->

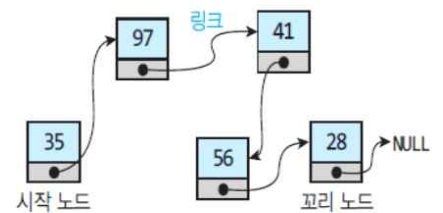
30->20->10->

30->20->10->40->

20->10->40->



(a) 배열 구조



(b) 연결된 구조

연결된 구조

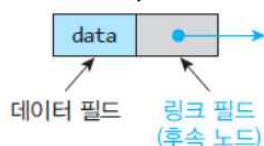
: 자료를 한군데 모아두는 것(배열구조)을 포기.

요소들이 메모리의 여기저기에 분산된 상태로 관리됨.

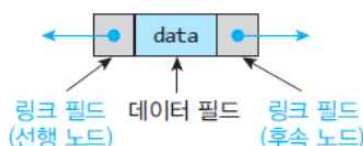
노드 = 데이터 필드 + 링크 필드

데이터 필드 (data field): 데이터 값을 저장

링크 필드 (link field): 다른 노드의 주소값을 저장 (포인터)



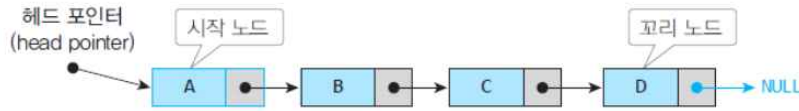
(a) 단순 연결을 위한 노드



(b) 이중 연결을 위한 노드

왼쪽은 singly, 오른쪽은 doubly linked list

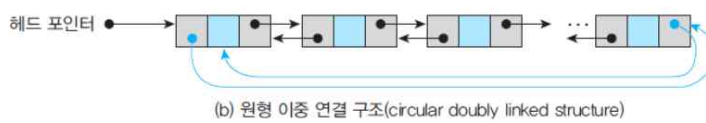
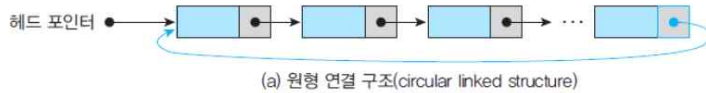
헤드 포인터 : 시작 노드의 주소를 저장하기 위한 포인터



□ 단순연결 (singly linked), 이중연결 (doubly linked)



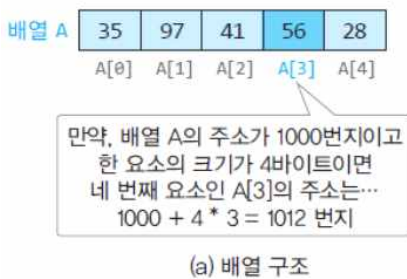
□ 원형 연결 (circular linked)



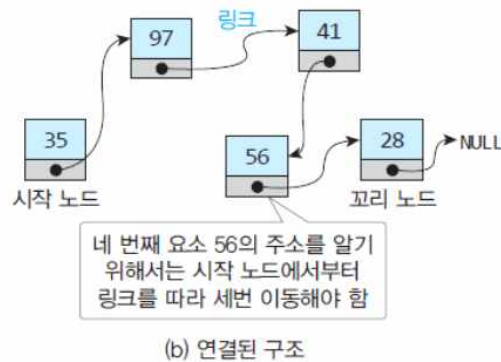
중앙대학교
JUNGANG UNIVERSITY

19

요소에 대한 접근



$O(1)$

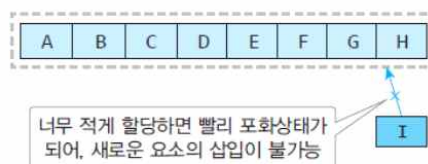
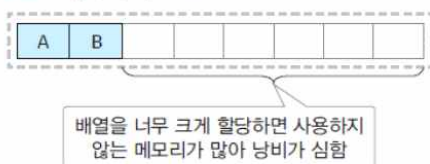


$O(n)$

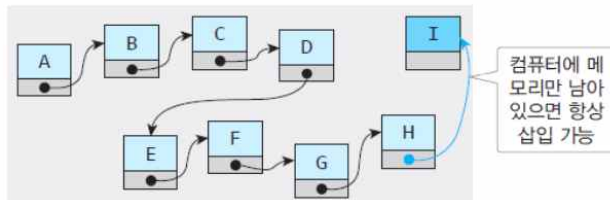
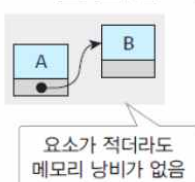
접근 : 배열이 빠르고, linked list가 느리다

용량적 관점

□ 배열구조

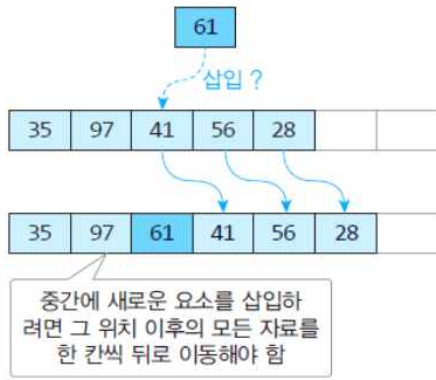


□ 연결된 구조



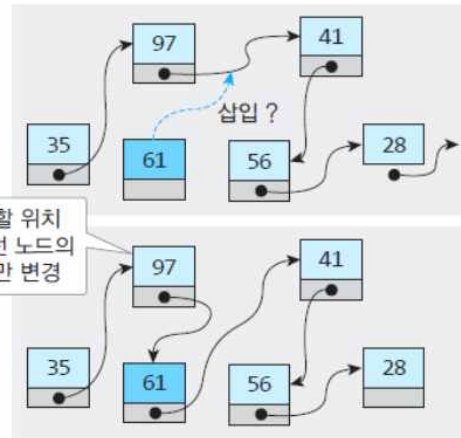
배열은 너무 크게 할당하면 낭비가 생기고, 너무 적게 할당하면 새로운 요소를 삽입할 수 없음.
Linked List는 요소가 적더라도 메모리 낭비가 없음. 컴퓨터에 메모리만 남아있으면 항상 삽입 가능.

중간에 자료 삽입



(a) 배열 구조의 삽입 연산

$O(n)$



(b) 연결된 구조의 삽입 연산

$O(1)$

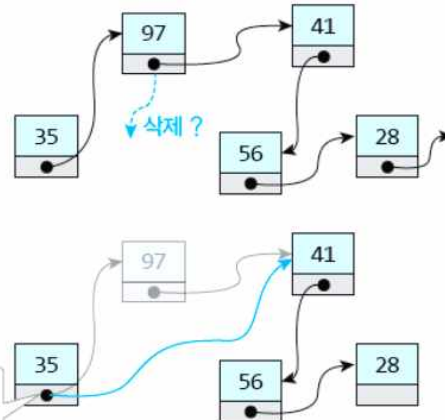
배열은 중간에 새로운 요소를 삽입하려면 그 위치 이후의 모든 자료를 한 칸씩 이동해야 함.
Linked List는 삽입할 위치 바로 전 노드의 링크만 변경하면 됨.

중간에 자료 삭제



(a) 배열 구조의 삭제 연산

$O(n)$



(b) 연결된 구조의 삭제 연산

$O(1)$

배열은 중간에 요소를 삭제하면 그 후의 모든 요소를 앞으로 한 칸씩 이동해야 함.
Linked List는 삭제할 위치 바로 전 노드의 링크만 변경하면 됨.

Linked(연결된) 구조의 장단점

□ 장점

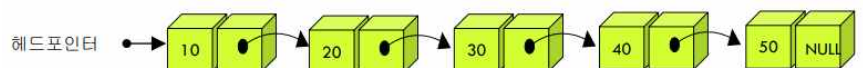
- 삽입, 삭제가 용이하다
- 연속된 메모리 공간이 필요없다
- 저장 공간 추가가 용이하다

□ 단점

- 구현이 어렵고 오류가 발생하기가 쉽다
- 데이터 탐색 시간이 오래 걸린다

단순 연결 리스트 Singlt linked list

- 하나의 링크 필드를 이용하여 연결된 리스트
- 마지막 노드의 링크 값은 NULL



- 정의 : 자기참조 구조체 이용
- 생성 : 동적 메모리 할당 (malloc())
- 삭제 : 동적 메모리 해제 (free())

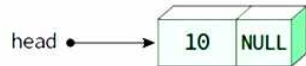
노드 정의

```
typedef int element;
typedef struct ListNode {    // 노드 타입을 자기참조 구조체 형태로 정의한다.
    element data;
    struct ListNode *link;
} ListNode;
```



리스트 생성

```
ListNode *head = (ListNode *)malloc(sizeof(ListNode));
head->data = 10;
head->link = NULL;
```



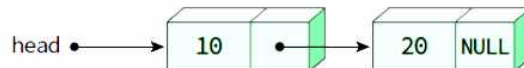
두번째 노드 생성

```
ListNode *p = (ListNode *)malloc(sizeof(ListNode));
p->data = 20;
p->link = NULL;
```



노드의 연결

```
head->link = p;
```

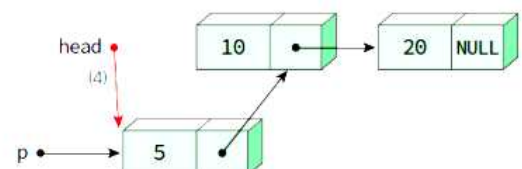
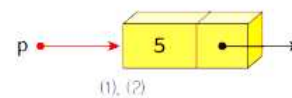
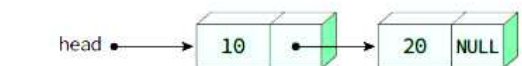


단순 연결 리스트 Singly linked list의 연산

insert_first()

```
ListNode* insert_first(ListNode *head, int value)
{
    ListNode *p =
        (ListNode *)malloc(sizeof(ListNode)); //(1)
    p->data = value; // (2)
    p->link = head; // (3)
    head = p; // (4)
    return head; // (5)
}
```

1. 동적 메모리 할당을 통해 새로운 노드 p를 생성
2. p의 데이터 필드에 value를 저장
3. p의 링크 필드를 현재 head 값으로 변경
4. head를 p값으로 변경
5. 변경된 head 포인터를 반환

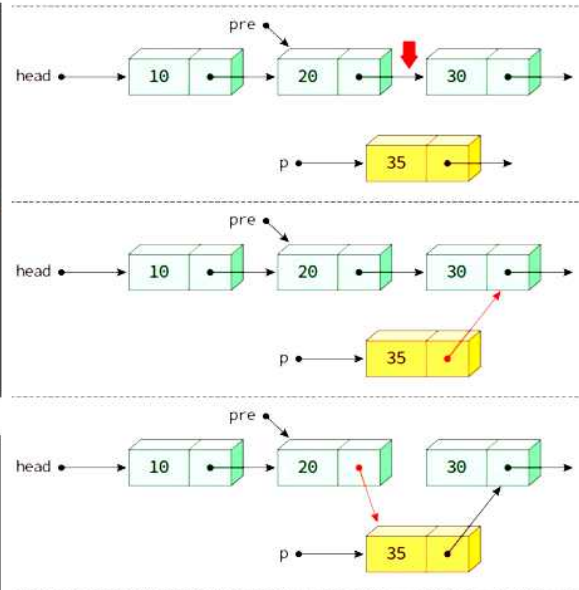


★변경된 head 포인터를 리턴한다!★

insert()

```
// 노드 pre 뒤에 새로운 노드 삽입
ListNode* insert(ListNode *head, ListNode *pre,
                  element value)
{
    ListNode *p =
        (ListNode *)malloc(sizeof(ListNode)); //(1)
    p->data = value;           //(2)
    p->link = pre->link;       //(3)
    pre->link = p;             //(4)
    return head;              //(5)
}
```

1. 동적 메모리 할당을 통해 새로운 노드 p를 생성
2. p의 데이터 필드에 value를 저장
3. p의 링크 필드가 pre의 링크 필드를 가리키게 변경
4. pre의 링크 필드가 p를 가리키도록 변경
5. 변경된 head 포인터 반환

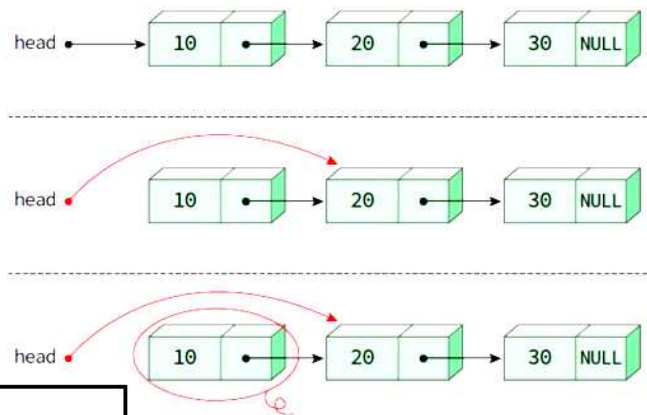


delete_first()

지울 때는 지우고자 하는 노드를 백업받아 놓아야 함!

```
ListNode* delete_first(ListNode *head)
{
    ListNode *removed; // (1)
    if (head == NULL) return NULL;
    removed = head;    // (2)
    head = removed->link; // (3)
    free(removed);      // (4)
    return head;        // (5)
}
```

1. 삭제할 노드를 가리킬 포인터 removed 생성
2. head 포인터의 값을 removed에 저장
3. head 포인터가 removed의 링크 필드를 가리키게 변경
4. removed가 가리키는 (삭제할) 노드의 동적 메모리 반환
5. 변경된 head 포인터 반환

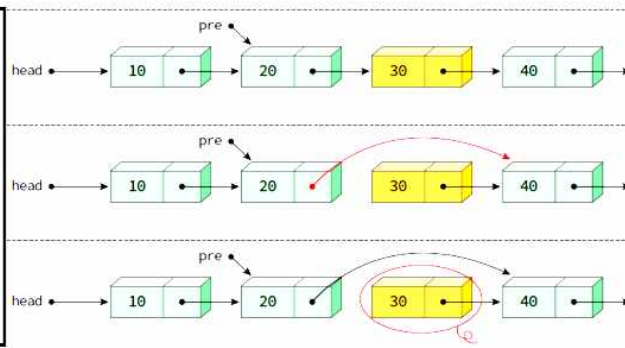


31

delete()

```
//pre가 가리키는 노드의 다음노드 삭제
ListNode* delete(ListNode *head,
                  ListNode *pre)
{
    ListNode *removed; // (1)
    removed = pre->link; // (2)
    pre->link = removed->link; // (3)
    free(removed);      // (4)
    return head;        // (5)
}
```

1. 삭제할 노드를 가리킬 removed 포인터 생성
2. 삭제할 노드를 removed 포인터가 가리키게 변경
3. 이전 노드가 삭제할 다음 노드를 가리키게 변경
4. 삭제할 (removed가 가리키는) 노드의 동적 메모리 반환
5. 변경된 head 포인터 반환



32

print_list()

```
void print_list(ListNode *head)
{
    ListNode *p;
    for (p = head; p != NULL; p = p->link)
        printf("%d->", p->data);
    printf("NULL \n");
}
```

search_list()

```
void print_list(ListNode *head)
{
    ListNode *p;
    for (p = head; p != NULL; p = p->link)
        printf("%d->", p->data);
    printf("NULL \n");
}
```

테스트 해보기

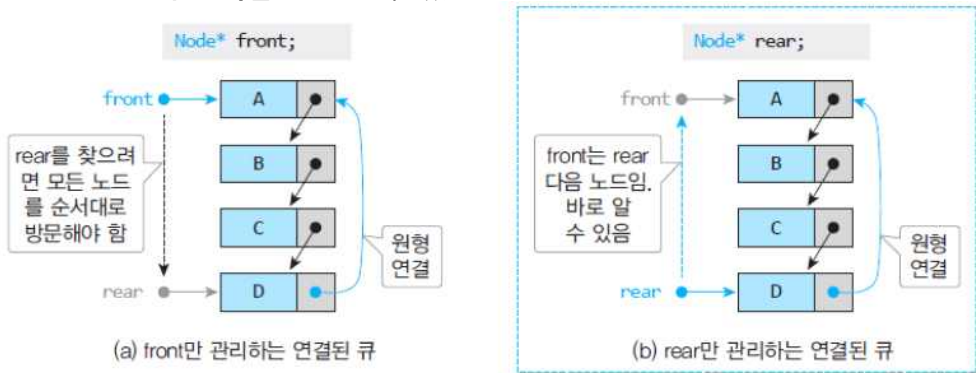
```
int main(void)
{
    ListNode *head = NULL;

    for (int i = 0; i < 5; i++) {
        head = insert_first(head, i);
        print_list(head);
    }
    for (int i = 0; i < 5; i++) {
        head = delete_first(head);
        print_list(head);
    }
    return 0;
}
```

0->NULL
1->0->NULL
2->1->0->NULL
3->2->1->0->NULL
4->3->2->1->0->NULL
3->2->1->0->NULL
2->1->0->NULL
1->0->NULL
0->NULL
NULL

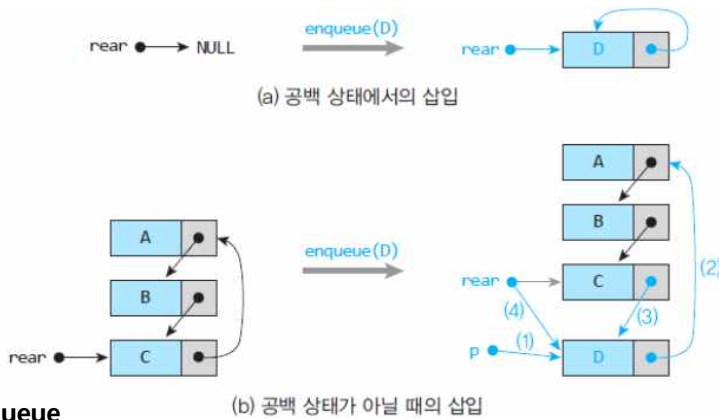
원형 링크드 리스트 큐 (Circular) Linked List Queue

Circular queue를 linked list 구조로 구현
front와 rear 중 하나만 관리하면 됨: 보통 **rear만 관리함**.
front 는 **rear** 바로 다음임을 알 수 있음



enqueue

rear에 새로운 요소를 삽입.



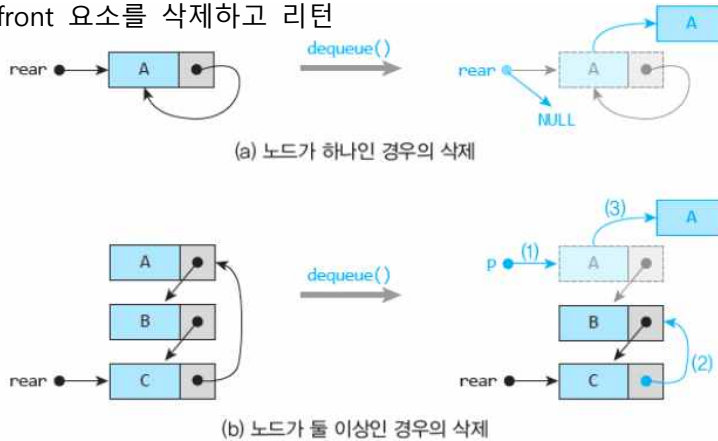
알고리즘 5.10 원형 연결 큐의

```

01 enqueue(e)
02   p ← alloc_node(e)
03   if is_empty() :
04       rear ← p
05       p.link ← p
06   else :
07       p.link ← rear.link
08       rear.link ← p
09       rear ← p
    
```

dequeue

front 요소를 삭제하고 리턴



알고리즘 5.11 원형 연결 큐의 삭제

```

01 dequeue()
02   if is_empty() :
03       error "underflow"
04   else :
05       p ← rear.link
06       if rear == p :
07           rear ← NULL
08       else :
09           rear.link ← p.link
10       return free_node(p)
    
```

Linked List Queue 실제 구현

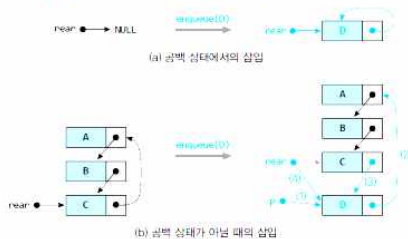
```

typedef int element;
typedef struct ListNode {    // 노드 타입을 자기참조 구조체로 정의
    element data;
    struct ListNode *link;
} ListNode;
ListNode *rear = NULL;      // rear 정보만 유지 (front 유지 불필요)
    
```

void enqueue (element e)

```

{
    ListNode *p = (ListNode *)malloc(sizeof(ListNode));
    p->data = e;
    p->link = NULL;
    if(is_empty()) {    // 이제는 full 인지 체크할 필요가 없음
        rear = p;
        p->link = p;
    } else {
        p->link = rear->link;
        rear->link = p;
        rear = p;
    }
}
    
```



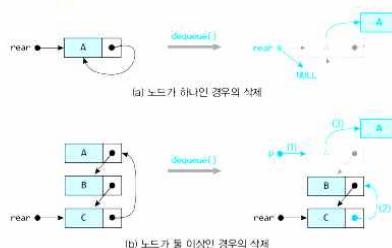
element dequeue ()

```

{
    if(is_empty()) {
        error("Underflow Error");
    }

    ListNode *p = rear->link; //삭제할 노드 (즉, front node)
    if(rear == p)    // 노드가 하나인 경우
        rear = NULL;
    else
        rear->link = p->link;

    element e = p->data;
    free(p); // front node 삭제
    return e;
}
    
```



```

//linked list queue의 총 요소의 수 구하기
int size()
{
    if(is_empty())
        return 0;
    int count = 1; //rear다음 (front)부터 시작하므로 rear 이미 count
    for (ListNode* p = rear->link; p != rear; p = p->link) //front 부터 시작
        count++;
    return count;
}
//모든 요소 삭제 (동적 메모리 해제)
void destroy_queue()
{
    while(!is_empty()) dequeue();
}

int is_empty() { return rear == NULL; }
int is_full() { return 0; }
int init_queue() { rear = NULL; }

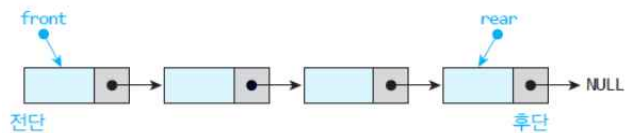
```

linked list에서는 full이라는 개념이 존재하지 않음!

Linked List Deque 링크드 리스트 덱

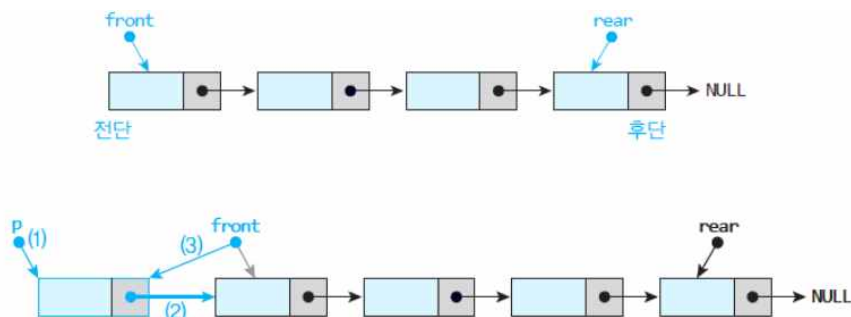
Deque: front와 rear에서 모두 삽입과 삭제가 가능한 자료구조
여기서는 circular가 아닌 linear 형태로 가정

front와 rear 모두 이용



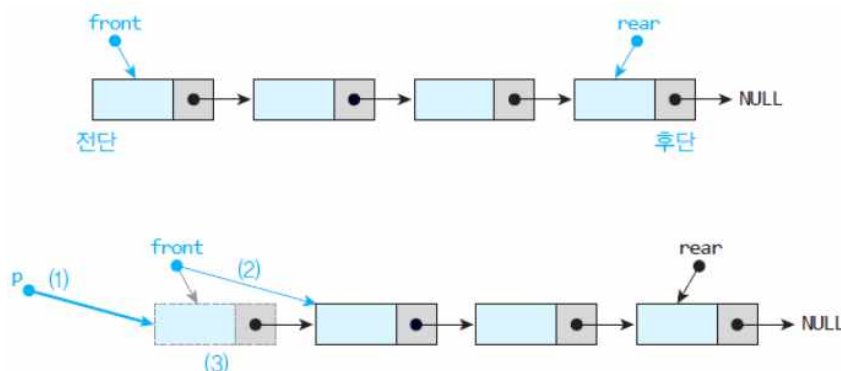
add_front(e)

요소를 저장할 노드 p 생성 -> p의 링크가 front를 가리킴 -> 이제는 front가 p를 가리킴



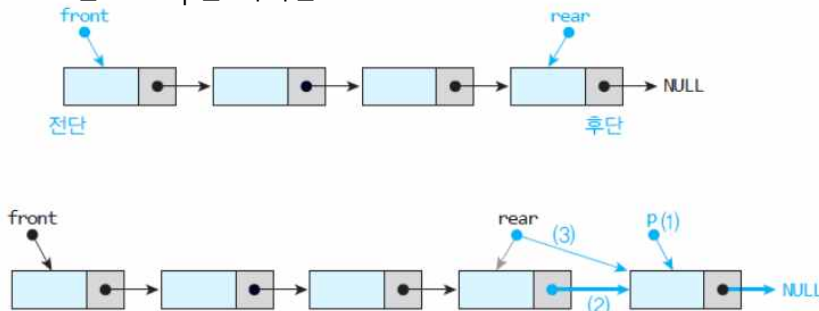
delete_front()

포인터 p가 삭제할 front를 가리킴 -> 이제 front의 다음 노드(front->link)를 가리킴
-> 노드 p 삭제하고 요소 반환



add_rear(e)

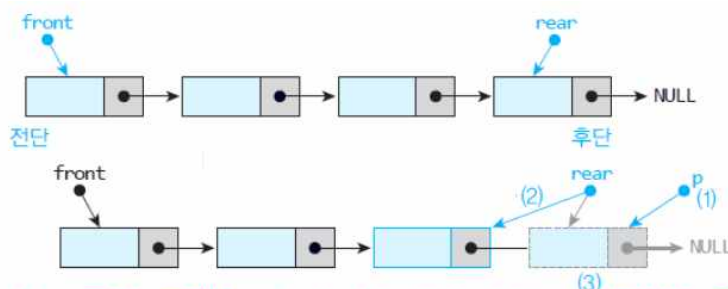
요소를 저장할 새로운 노드 p 생성 -> rear의 링크(rear->link)가 노드 p를 가리킴
-> rear는 노드 p를 가리킴



delete_rear()

포인터 p가 삭제할 노드(rear)를 가리킴 -> 이제 **rear는 선행 노드**를 가리킴 -> 노드p(즉, rear)를 삭제하고 요소를 반환

★그런데 여기서 rear가 가리켜야 하는 선행 노드를 알 수 없음!★



O(1) 구현 불가능 -> doubly linked list 사용해야 함

7장 연결 리스트2

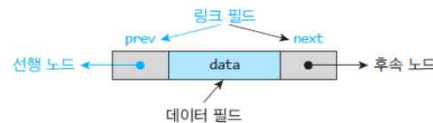
이중 연결 리스트 Doubly Linked List

단순 연결 리스트(singly linked list)의 단점 : 선행 노드를 찾기가 힘들다!

이중 연결 리스트

: 하나의 노드가 선행 노드와 후속 노드에 대한 두 개의 링크를 동시에 가지는 리스트

단점 : 공간을 많이 차지하고 코드가 복잡함.



노드 구조

노드 = 데이터 필드 + 두 개의 링크 필드(선행/후속)

```
typedef struct DNode {
    Element data; // 데이터 필드
    struct DNode* prev; // 선행 노드
    struct DNode* next; // 후속 노드(Node의 link와 동일)
} DNode;
```

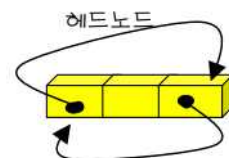
p == p->next->prev == p->prev->next

헤드 노드 head node

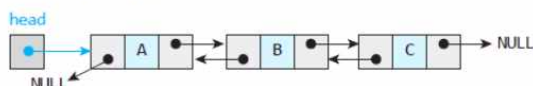
데이터를 가지지 않고 단지 삽입/삭제 코드를 간단하게 할 목적으로 만들어진 노드.

Empty 상태에서는 헤드 노드만 존재한다.

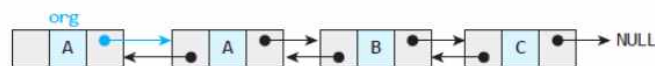
역할은 헤드 포인터와 동일! 사실 헤드 포인터를 써도 기능상 문제는 없음.



헤드 포인터 vs. 헤드 노드



(a) 이중 연결 리스트(헤드 포인터 head 사용)



(b) 이중 연결 리스트(헤드 노드 org 사용)

get_node(pos)

연결 리스트에서 **pos번째 노드의 주소(포인터)를 반환**하는 함수
예를 들어 get_node(7) 이면 7번째 노드의 주소를 리턴.

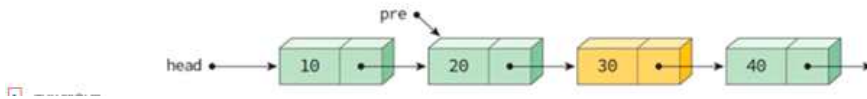
알고리즘 6.14 이중 연결 리스트의 pos 위치의 노드 찾기

```

01 get_node(pos)
02   p ← org의 주소      // -1인 위치(헤드 노드 주소)에서 출발
03   for i ← 0 to pos :  // pos+1번 이동해야 함
04       if p = NULL :
05           return NULL
06       else : p ← p.next // link 대신에 next 사용해야 함
07   return p

```

참고: 단순 연결 리스트에서 *pre 포인터의 역할



insert(pos, e)

pos 위치에 요소 e를 삽입.



알고리즘 6.15 이중 연결 리스트의 삽입 연산

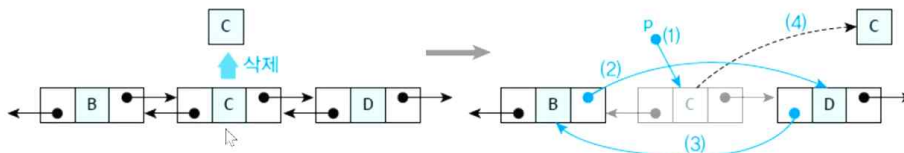
```

01 insert(pos, e)
02   before ← get_node(pos-1) O(n) // 그림 6.18의 (1) 삽입할 위치의 이전 노드 before를 찾는다
03   if before = NULL :
04       error "Invalid Position Error"
05   else :
06       p ← alloc_dnode(e)      // 그림 6.18의 (2) 새로운 노드 p를 동적으로 생성한다
07       p.next ← before.next    // 그림 6.18의 (3) p의 next가 before의 후속 노드를 가리킨다
08       p.prev ← before         // 그림 6.18의 (4) p의 prev가 before를 가리킨다
09       before.next ← p        // 그림 6.18의 (5) p의 next가 p를 가리킨다
10       if p.next != NULL :
11           p.next->prev ← p    // 그림 6.18의 (6) 후속 노드의 prev가 p를 가리킨다

```

delete(pos)

pos 위치의 요소를 꺼내서 반환



알고리즘 6.16 이중 연결 리스트의 삭제 연산

```

01 delete(pos)
02   p ← get_node(pos)      // 그림 6.19의 (1)
03   if p = NULL :
04       error "Invalid Position Error"
05   p.prev.next ← p.next    // 그림 6.18의 (2)
06   if p.next != NULL :
07       p.next.prev ← p.prev // 그림 6.19의 (3)
08   return free_node(e)     // 그림 6.19의 (4)

```

doubly linked list 실제 코드

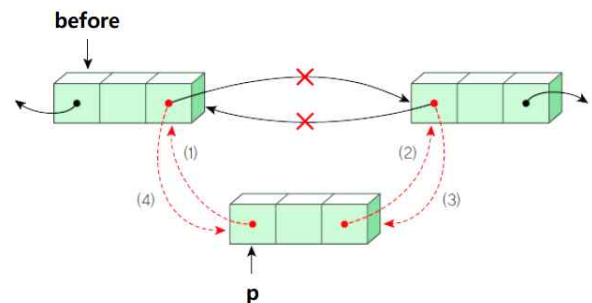
```
typedef struct DNode {
    Element data;
    struct DNode* prev;
    struct DNode* next;
} DNode;

DNode head;          // head node

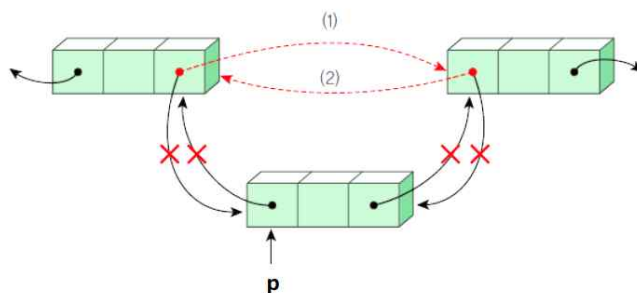
DNode* get_node(int pos)
{
    DNode* p = &head;
    for(int i = 0; i <= pos; i++)
        if(p == NULL) return NULL;
        else p = p->next;
    return p;
}
```

```
int is_empty { return (head.next == NULL); }
int is_full { return 0; }
void init_list() { head.next = NULL; }
```

```
void insert(int pos, Element data)
{
    DNode* before = get_node(pos - 1);
    DNode* p = (DNode *)malloc(sizeof(DNode));
    p->next = before->next;
    p->prev = before;
    before->next->pre = p; // p->next->prev = p;
    before->next = p;
}
```



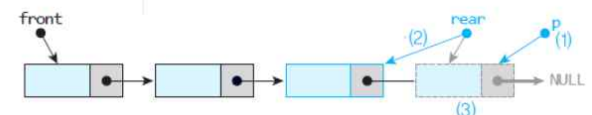
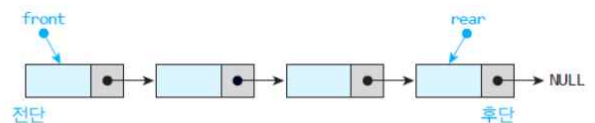
```
void delete(int pos)
{
    DNode* p = get_node(pos);
    p->prev->next = p->next;
    p->next->prev = p->prev;
    free(p);
}
```



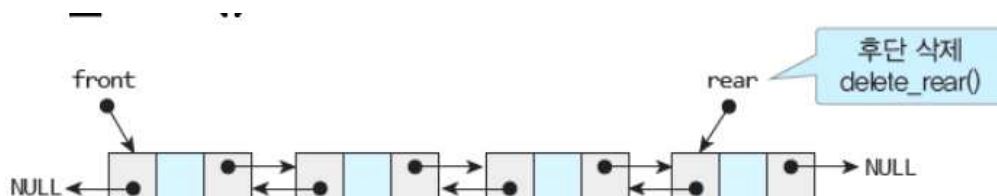
singly linked list queue에서 delete_rear()를 구현

포인터 p가 삭제할 노드(rear)를 가리킴
 rear는 선행 노드를 가리켜야 하는데, 선행 노드를 바로 알 수 없음
 노드 p(즉, rear)를 삭제하고 요소를 반환

>> Doubly linked list를 사용해야 한다!



O(1) 구현 불가능 → doubly linked list 사용해야 함





(a) 노드가 하나인 경우의 후단 삭제



(b) 노드가 둘 이상인 경우의 후단 삭제

```

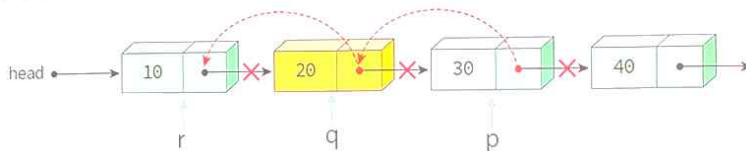
Element delete_rear()
{
    if(is_empty())
        error("Underflow Error");

    DNode* p = rear;
    if (front == rear)    //요소가 하나인 경우
        front = rear = NULL;
    else {
        rear = rear->prev;  ← 선행 노드의 링크가 존재: O(1)
        rear->next = NULL;
    }
    Element e = p->data;
    free (p);
    return e;
}

```

Lab : 단순 연결 리스트를 역순으로 변경

- 새로운 연결 리스트를 만들지 않고 해결
- 리스트를 순회하면서 링크의 방향을 역순으로 변경
- 힌트: 역순으로 바꾸기 전에 미리 뒤의 노드를 알아야함



- 3개 포인터 사용
 - r: 이미 역순으로 변경된 노드
 - q: 현재 역순으로 변경할 노드
 - p: 현재 역순으로 변경할 노드의 뒤 노드
- 현재 q가 가리키는 노드

ListNode* reverse(ListNode *head)

```

{
    // 순회 포인터로 p, q, r을 사용
    ListNode *p, *q, *r;

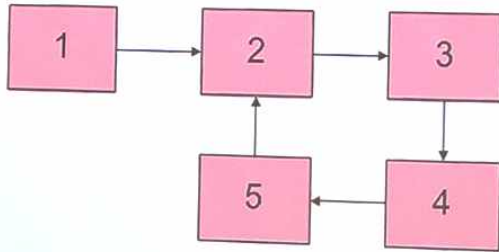
    p = head;    // p는 역순으로 만들 리스트
    q = NULL;    // q는 역순으로 만들 노드
    while (p != NULL) {
        r = q;    // r은 역순으로 된 리스트.
                  // r은 q, q는 p를 차례로 따라간다.

        q = p;
        p = p->link;    // p는 다음 노드를 저장한다
        q->link = r;    // q의 링크 방향을 바꾼다.
    }
    return q;
}

```

연결 리스트에서 Loop (cycle)

- Detect a loop ---- 1단계
- Remove the link ----- 2단계



Loop Detection 방법 1. 플래그로 체크

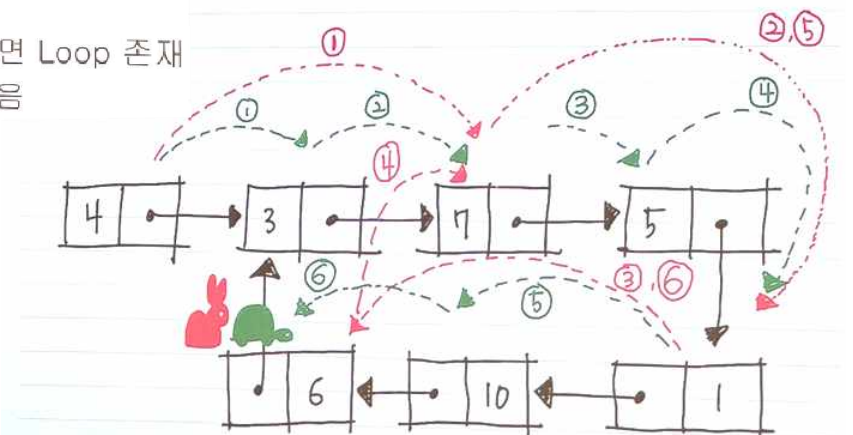
- 노드에 flag를 추가하여 방문마다 표시
- 만약 flag가 1이면 Loop이 존재함

```
typedef struct Node {
    int data;
    struct Node* next;
    int flag;          //노드 방문을 체크하기 위한 플래그
} ListNode;

int detectLoop(ListNode* head)
{
    while (head != NULL) {
        if (head->flag == 1) //이미 노드를 방문한 적이 있으면 Loop
            return true;
        head->flag = 1;      //노드를 방문할 때마다 flag에 표시
        head = head->next;
    }
    return false;
}
```

Loop Detection 방법 2. Floyd's cycle detection - 거북이와 토끼

- Linked list를 두개의 포인터를 이용하여 순회
 - Fast pointer: 한 번에 두 개의 노드씩 이동
 - Slow pointer: 한 번에 한 개의 노드씩 이동
- 만약 두 포인터가 같은 노드에서 만나면 Loop 존재
 - 그렇지 않으면 Loop가 존재하지 않음



```

void detectAndRemoveLoop(ListNode* head)
{
    if (head == NULL || head->next == NULL) //empty or only one node
        return;

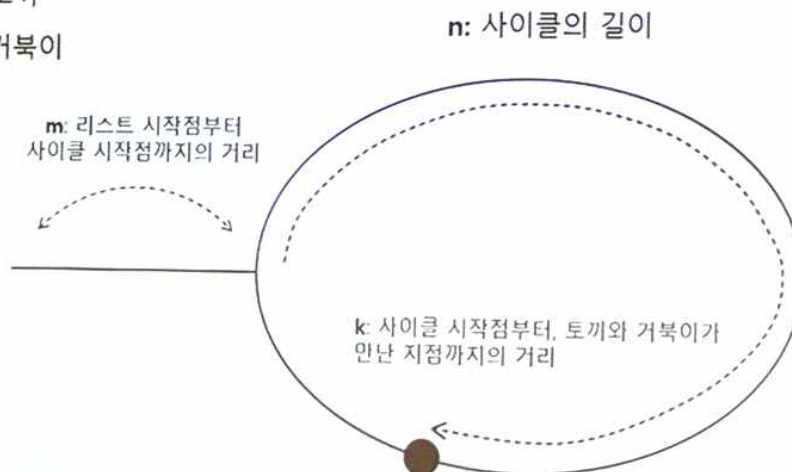
    ListNode *slow = *fast = head;
    while (slow && fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) {
            break;
        }
    }
    if (slow == fast) { //loop이 존재하면
        slow = head;
        while (slow->next != fast->next) { //loop 포인트 시작점 찾기
            slow = slow->next;
            fast = fast->next;
        }
        fast->next = NULL; // loop을 제거
    }
}

```

loop를 찾으면 거북이를 head로 보내놓고 loop의 시작점을 찾은 후에 loop를 제거!

□ 동작 원리

- 토끼
- 거북이



토끼가 사이클을 a번 돌고, 거북이가 사이클을 b번 돈 뒤, 서로 만났을 때

- 토끼의 이동 거리: $m + a * n + k$
- 거북이의 이동 거리: $m + b * n + k$

□ 동작 원리

- 거북이의 이동거리 * 2 = 토끼의 이동거리
- $(m + b * n + k) * 2 = (m + a * n + k)$
- $m + k = (a - 2b) * n$

→ 첫 시작점에서 사이클 시작점을 지나 서로 만난곳까지 가는 거리 = 사이클을 $(a - 2b)$ 번 도는 거리

□ $m = (a - 2b) * n - k$

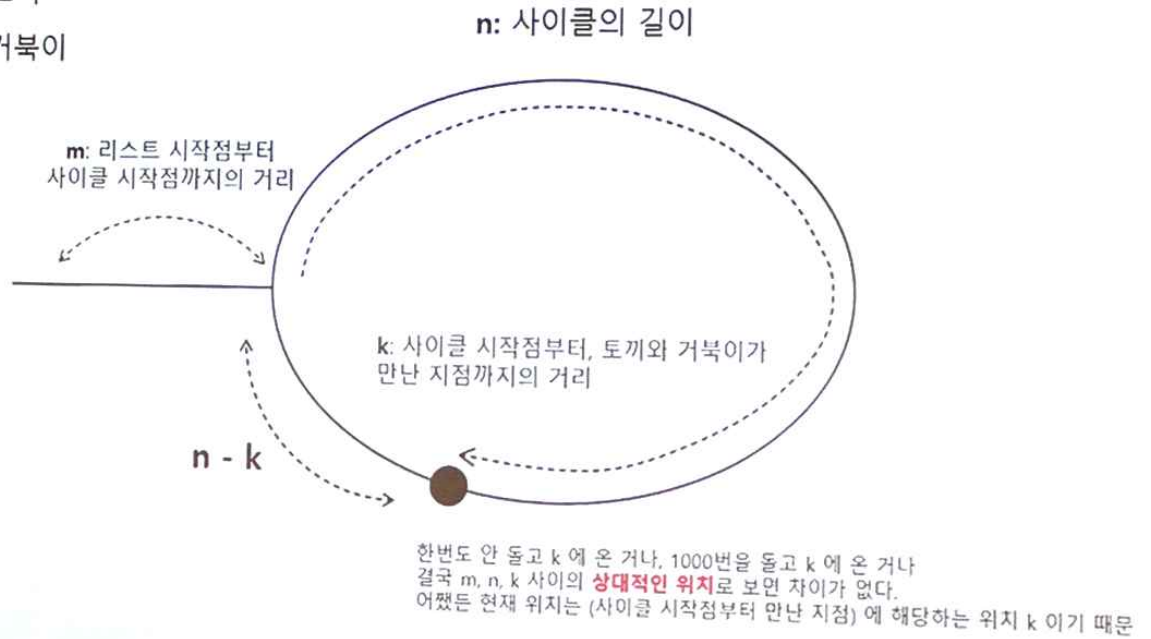
→ 사이클을 몇 번 돌든 총 이동거리는 바뀌지만 상대적인 현재 위치는 항상 동일

→ 결국 $(a - 2b)$ 값은 거리가 아닌 상대적인 위치 입장에서 보면 의미가 없어지게 됨

→ 우리에게 필요한 사실은 결국 k 위치 (토끼와 거북이가 만난점) 에서 $(n - k)$ 만큼 이동하면 사이클 시작점으로 돌아가게 된다는 점임

□ 동작 원리

- 토끼
- 거북이



1칸씩 이동할 때 m 만큼 간 거리와 $(n - k)$ 만큼 이동한 거리가 동일하므로 결국 거북이를 원점으로 돌려놓은 뒤 토끼와 거북이가 둘 다 1칸씩 이동하면 사이클 시작점에서 만나게 된다는 뜻.

□ 핵심 개념:

토끼가 더 빨리 움직이니까, 언젠가는 거북이를 따라잡아요. 이걸 수학적으로 분석하면:

1. 토끼는 거북이보다 매번 1칸씩 더 빨리 움직이므로, 결국 사이클 안에서 같은 지점에서 만나게 돼요.
2. 만나고 나서 둘 다 속도를 1칸씩으로 맞춰서 다시 출발하면,
3. 리스트의 시작점에서부터 **거북이는 m칸**, 현재 거북이와 토끼는 $(n - k)$ 칸 가게 되고,
4. 둘은 사이클 시작점에서 다시 만나게 돼요

"결국 거북이를 원점으로 돌려놓은 뒤 토끼와 거북이 둘 다 1칸씩 이동하면 사이클 시작점에서 만나게 된다는 뜻."

거북이 원점 복귀: 거북이를 리스트의 시작점(head)으로 다시 보냅니다.

토끼는 만난 지점에 유지: 토끼는 두 포인터가 처음 만났던 그 지점(k)에 그대로 둡니다.

동시에 1칸씩 이동: 이제 거북이와 토끼를 동시에 한 칸씩 이동시킵니다.

m 과 $n - k$ 는 (사이클 길이 n 의 정수배만큼) 동일한 거리를 의미합니다.

따라서 거북이가 시작점에서 m 칸을 이동하는 동안, 토끼도 만난 지점에서 m 칸 (이는 사이클 내에서 $n - k$ 와 같은 지점)을 이동하면, 두 포인터는 정확히 사이클의 시작점에서 만나게 됩니다.

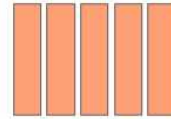
8장 트리

트리(List)란?

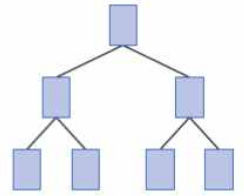
계층적인 구조를 나타내는 자료구조

리스트, 스택, 큐 등은 선형구조

트리는 부모-자식 관계의 노드들로 구성 => 비선형이다!



선형 자료구조



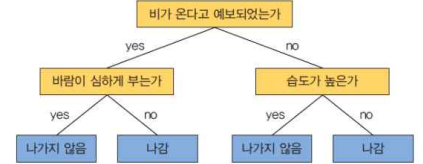
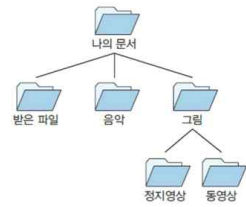
비선형 자료구조

응용분야

회사의 계층적인 조직도

컴퓨터 디스크의 디렉토리 구조

인공지능에서 의사결정트리 (decision tree)



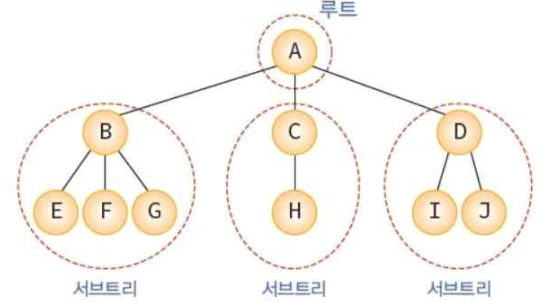
노드(node) : 트리의 구성요소

루트(root) : 부모가 없는 최상위 노드

서브트리(subtree) : 하나의 노드와 그 자손들로 이루어진 트리

단말노드(terminal node, or leaf node) : 자식이 없는 노드

비단말노드 : 적어도 하나의 자식이 있는 노드

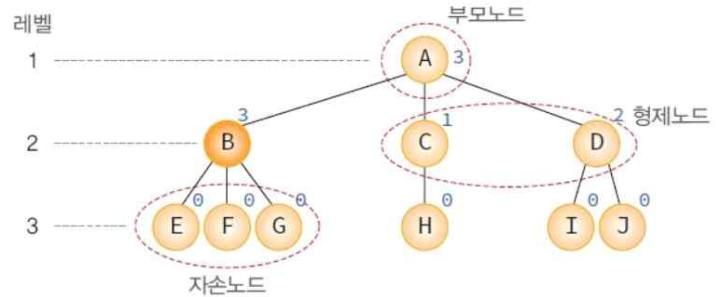


레벨(level) : 트리의 각층의 번호

높이(height) : 트리의 최대 레벨

차수(degree) : 노드가 가지고 있는 자식 노드의 개수

자식, 부모, 형제, 조상, 자손 노드: 인간과 동일한 개념



트리의 종류

일반 트리

이진 트리 : 모든 노드가 서브트리 2개!

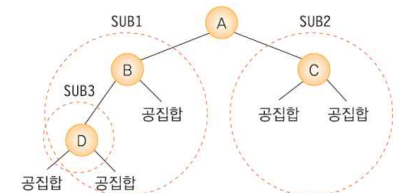
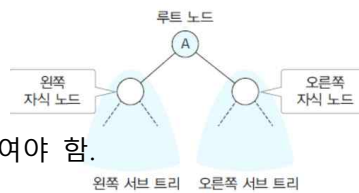
이진 트리

: 모든 노드가 2개의 서브 트리를 가지고 있는 트리

* 서브 트리는 공집합일 수 있음

* 각 노드는 최대 2개까지 자식 노드가 존재

* 이진 트리들의 서브 트리들은 모두 이진 트리여야 함.



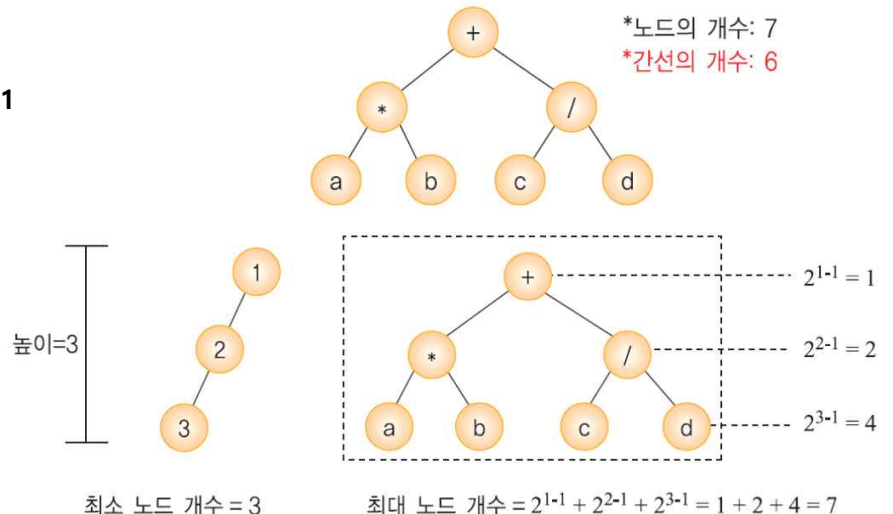
이진 트리의 성질

노드의 개수가 n개이면 간선의 개수는 n-1

높이가 h인 이진 트리의 경우,

최소 h개의 노드

최대 $2^h - 1$ 개의 노드



n개의 노드 가지는 이진 트리의 높이

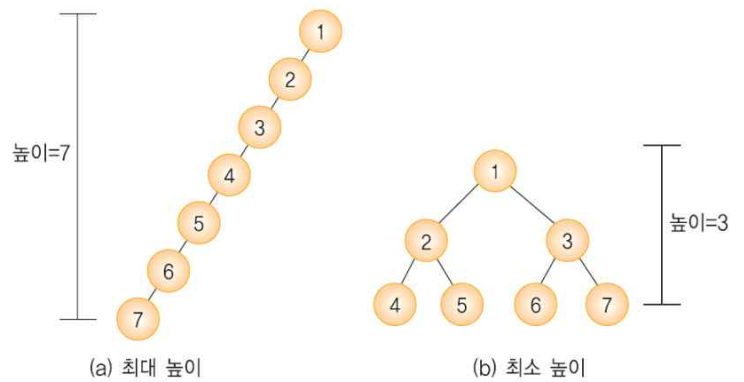
최대 : n

최소 $\lceil \log_2 (n+1) \rceil$

이진 트리의 분류

포화 이진 트리 (full binary tree)

모든 노드가 0개나 또는 2개의 자식 노드만을 가지고 있는 이진 트리



완벽 이진 트리 (perfect binary tree)

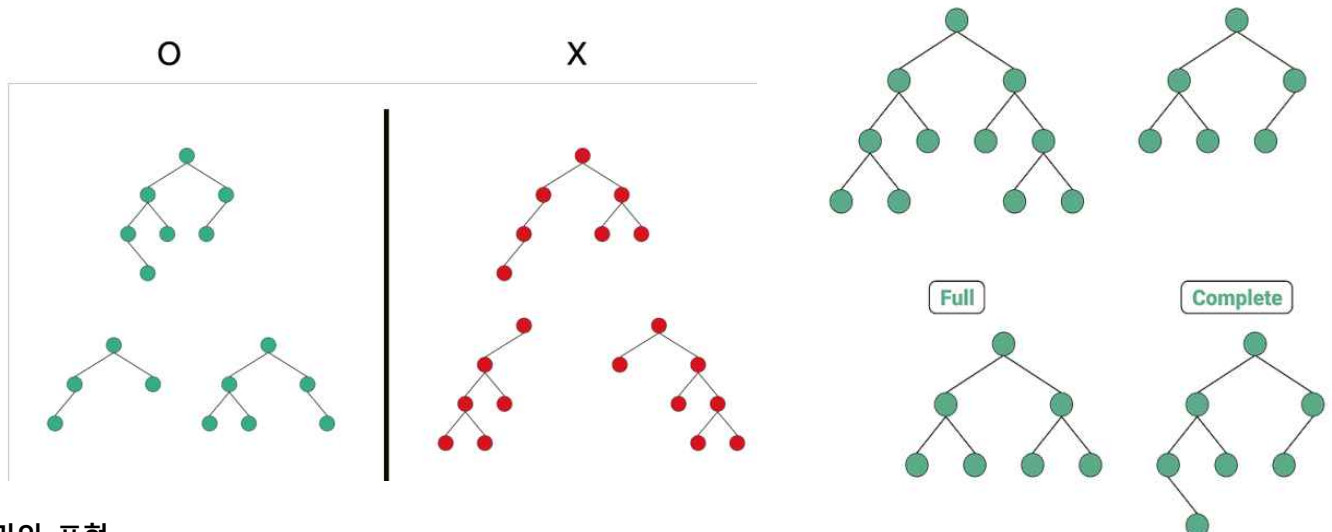
트리의 각 레벨에 노드가 가득 차 있는 이진 트리

완전 이진 트리 (complete binary tree)

레벨 1부터 마지막 레벨 직전(k-1)레벨까지 노드가 모두 채워져 있고, 마지막 레벨 k에서는 왼쪽부터 오른쪽으로 노드가 순서대로 채워져 있는 이진 트리

균형 이진 트리 (balanced binary tree)

모든 노드의 왼쪽과 오른쪽 서브 트리의 높이 차이가 최대 1을 초과할 수 없는 이진 트리

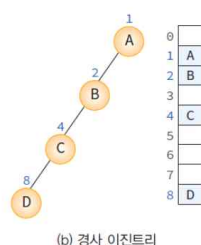
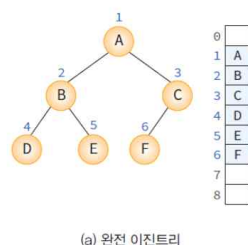


이진 트리의 표현

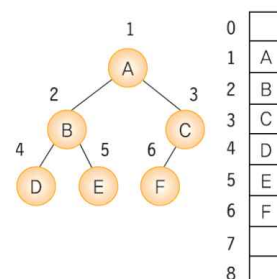
배열 표현법 : 배열을 이용하는 방법

저장하고자 하는 이진 트리의 높이가 k인 경우 최대 $2^k - 1$ 개의 공간을 할당한 다음 complete binary tree 번호대로 노드를 저장.

모든 이진 트리를 **complete binary tree** 라고 가정하고 각 노드에 번호를 붙여서 그 번호의 배열의 인덱스로 삼아 노드의 데이터를 배열에 저장하는 방법



```
typedef char TreeElement;
TreeElement t1[MAX_SIZE] = { 0, 'A', 'B', 'C', 'D', 'E', 'F' };
TreeElement t2[MAX_SIZE] = { 0, 'A', 'B', 0, 'C', 0, 0, 'D' };
// 중앙에 0이 들어가는 것
```



부모와 자식 인덱스 관계

- * 노드 i의 부모 노드 인덱스 = $i/2$
- * 노드 i의 왼쪽 자식 노드 인덱스 = $2i$
- * 노드 i의 오른쪽 자식 노드 인덱스 = $2i+1$

링크 표현법 : 포인터(링크)를 이용하는 방법

각 노드가 포인터들을 가지고 있는 구조체로 표현되어 노드와 노드를 링크 필드로 연결하는 방법

노드는 구조체로 표현

링크는 포인터로 표현

```
typedef struct TreeNode {
    int data;
    struct TreeNode *left;
    struct TreeNode *right;
} TNode;
```

```
// n1
// / |
// n2 n3
```

```
void main()
```

```
{
```

```
    TNode *n1, *n2, *n3;
```

```
    n1= (TNode *)malloc(sizeof(TNode));
```

```
    n2= (TNode *)malloc(sizeof(TNode));
```

```
    n3= (TNode *)malloc(sizeof(TNode));
```

```
    n1->data = 10;           // 첫 번째 노드 설정
```

```
    n1->left = n2;
```

```
    n1->right = n3;
```

```
    n2->data = 20;           // 두 번째 노드 설정
```

```
    n2->left = NULL;
```

```
    n2->right = NULL;
```

```
    n3->data = 30;           // 세 번째 노드 설정
```

```
    n3->left = NULL;
```

```
    n3->right = NULL;
```

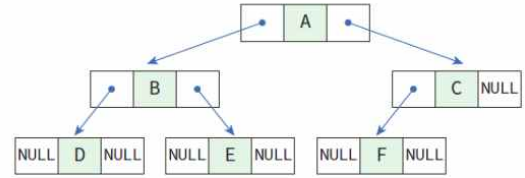
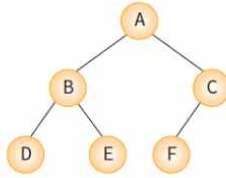
```
    free(n1);
```

```
    free(n2);
```

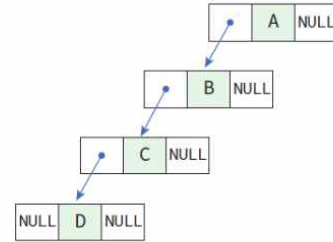
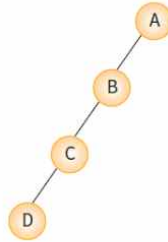
```
    free(n3);
```

```
    return 0;
```

```
}
```



(a) 완전 이진트리



(b) 경사 이진트리

링크의 동적 생성과 해제

□ 트리의 동적 생성 함수: `create_tree(data, left, right)`

```
TNode* create_tree(TreeElement data, TNode* left, TNode* right)
{
    TNode* n = (TNode*)malloc(sizeof(TNode));
    n->data = data;
    n->left = left;
    n->right = right;
    return n;
}
```

□ 트리의 동적 해제 함수: `delete_tree(n)`

- 좌/우 서브 트리를 삭제 후 해당 노드 (n)를 삭제
- 순환 호출

```
typedef struct TreeNode {
    int data;
    struct TreeNode *left;
    struct TreeNode *right;
} TNode;
```

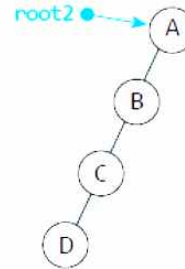
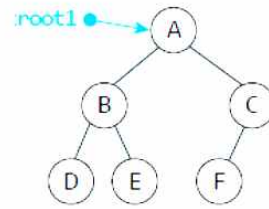
```
void delete_tree(TNode* n)
{
    if(n != NULL) {
        delete_tree(n->left);
        delete_tree(n->right);
        free(n);
    }
}
```

링크 표현법 예제

```

29 void main()
30 {
31     /* 그림 7.17(a) 트리 동적 생성 */
32     TNode *d = create_tree('D', NULL, NULL );
33     TNode *e = create_tree('E', NULL, NULL );
34     TNode *b = create_tree('B', d, e );
35     TNode *f = create_tree('F', NULL, NULL );
36     TNode *c = create_tree('C', f, NULL );
37     TNode *root1 = create_tree('A', b, c );
38
39     /* 그림 7.17(b) 트리 동적 생성 */
40     TNode* D = create_tree('D', NULL, NULL);
41     TNode* C = create_tree('C', D, NULL);
42     TNode* B = create_tree('B', C, NULL);
43     TNode* root2 = create_tree('A', B, NULL);
44
45     /* 트리 동적 해제 */
46     delete_tree(root1);
47     delete_tree(root2);
48 }

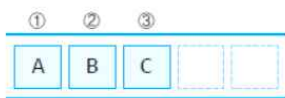
```



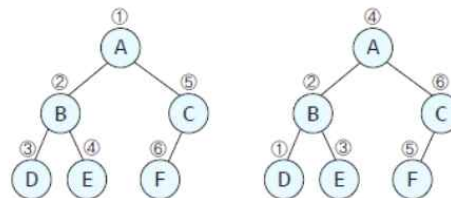
이진 트리의 순회(traversal)

순회(traversal) : 트리의 모든 노드들을 체계적으로 한 번씩 방문하는 작업

트리는 순회 방법이 다양. <-> 선형 자료구조는 순회 방법이 단순.



(a) 선형 자료 구조는 순회 방법이 단순함



(b) 트리는 순회 방법이 다양함

전위순회(preorder traversal) : VLR

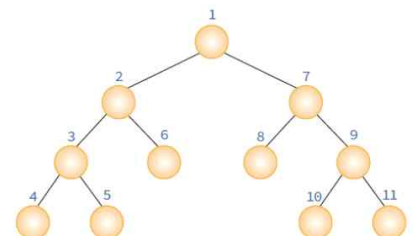
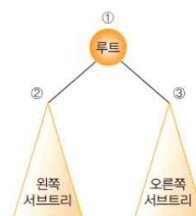
root -> **left** subtree -> **right** subtree

예) 구조화된 문서 출력

중위순회(inorder traversal) : LVR

left subtree -> **root** -> **right** subtree

예) 수식 트리



알고리즘 7.1 이진 트리의 전위순회

```

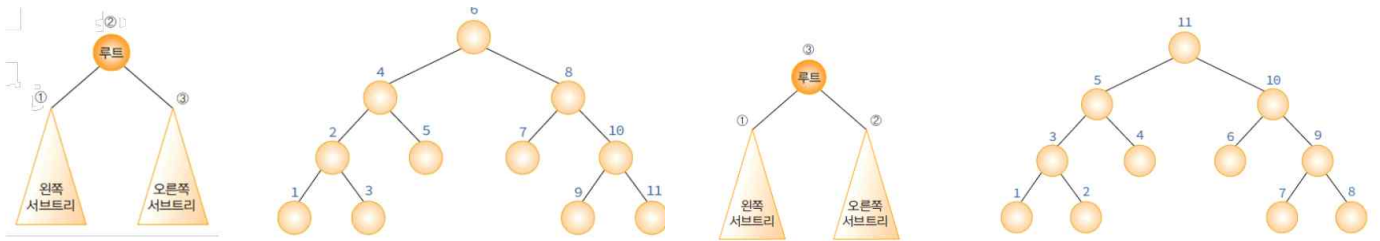
01 preorder(n)
02     if n ≠ NULL :           // 루트 n이 NULL이 아닐 때만 처리
03         VisitNode(n)       // 루트 노드 처리
04         preorder(n.left)    // 왼쪽 서브 트리 처리
05         preorder(n.right)   // 오른쪽 서브 트리 처리

```

후위순회(postorder traversal) : LRV

left subtree -> **right** subtree -> **root**

예) 컴퓨터 디렉토리 용량 계산



알고리즘 7.2 이진 트리의 중위 순회

```

01 inorder(n)
02     if n ≠ NULL :
03         inorder(n.left) // 왼쪽 서브 트리 처리
04         VisitNode(n)    // 루트 노드 처리
05         inorder(n.right) // 오른쪽 서브 트리 처리

```

알고리즘 7.3 이진 트리의 후위 순회

```

01 postorder(n)
02     if n ≠ NULL :
03         postorder(n.left) // 왼쪽 서브 트리 처리
04         postorder(n.right) // 오른쪽 서브 트리 처리
05         VisitNode(n)     // 루트 노드 처리

```

순회 프로그램 예제

```

typedef struct TreeNode {
    int data;
    struct TreeNode *left;
    struct TreeNode *right;
} TNode;

```

```

//          15
//      4      20
//  1      16      25

```

```

TNode n1={1, NULL, NULL};
TNode n2={4, &n1, NULL};
TNode n3={16, NULL, NULL};
TNode n4={25, NULL, NULL};
TNode n5={20, &n3, &n4};
TNode n6={15, &n2, &n5};
TNode *root= &n6;

```

```

int main(void)
{
    printf("중위 순회=");
    inorder(root);
    printf("\n");

    printf("전위 순회=");
    preorder(root);
    printf("\n");

    printf("후위 순회=");
    postorder(root);
    printf("\n");
    return 0;
}

```

```

void inorder( TNode *root ){
    if ( root ){
        inorder( root->left ); // 왼쪽 서브 트리 순회
        printf("%d", root->data ); // 노드 방문
        inorder( root->right ); // 오른쪽 서브 트리 순회
    }
}

void preorder( TNode *root ){
    if ( root ){
        printf("%d", root->data ); // 노드 방문
        preorder( root->left ); // 왼쪽 서브 트리 순회
        preorder( root->right ); // 오른쪽 서브 트리 순회
    }
}

void postorder( TNode *root ){
    if ( root ){
        postorder( root->left ); // 왼쪽 서브 트리 순회
        postorder( root->right ); // 오른쪽 서브 트리 순회
        printf("%d", root->data ); // 노드 방문
    }
}

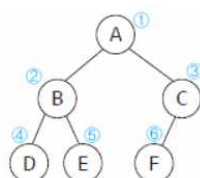
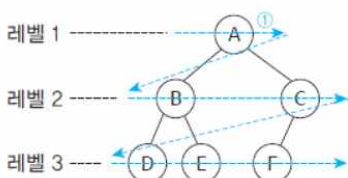
```

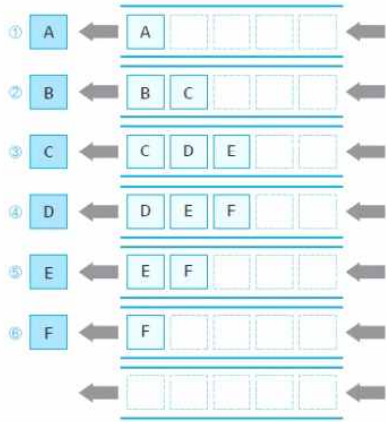
출력 : 중위 순회=[1] [4] [15] [16] [20] [25]
 전위 순회=[15] [4] [1] [20] [16] [25]
 후위 순회=[1] [4] [16] [25] [20] [15]

레벨 순회 Level Order Traversal

: 각 노드를 레벨 순으로 순회하는 방법.

레벨 순회는 **큐**를 사용함. <-> 다른 순회는 **스택** 사용





알고리즘 7.4

이진 트리의 레벨 순회

```

01 levelorder(root)
02   if root ≠ NULL :
03       init_queue()
04       enqueue(root)
05       while not is_empty(queue) :
06           n ← dequeue()
07           if n ≠ NULL :
08               VisitNode(n)
09               enqueue(n.left)
10               enqueue(n.right)

```

꼬집어 낼 때마다 자식 노드들을 enqueue한다.

2. A를 꼬집어내면 B, C를 enqueue

3. B를 꼬집어내면 자식노드인 D, E를 enqueue한다

```
#include "CircularQueue.h"
```

```
void levelorder(TNode *root)
```

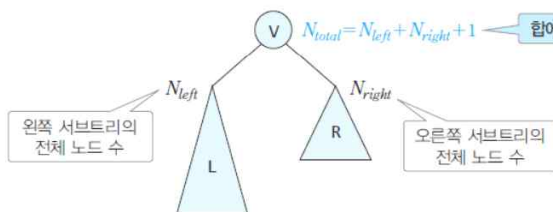
```

{
    if (root != NULL) {
        init_queue();
        enqueue(root);
        while(!is_empty()) {
            TNode* n = dequeue();
            if(n != NULL) {
                printf("[%d]", root->data);
                enqueue(n->left);
                enqueue(n->right);
            }
        }
    }
}

```

이진 트리 연산

노드 개수 구하기 : 각각 좌우 서브 트리에 대해 순환호출 한 후 반환값에 +1



알고리즘 7.5

이진 트리의 노드 수를 구하기

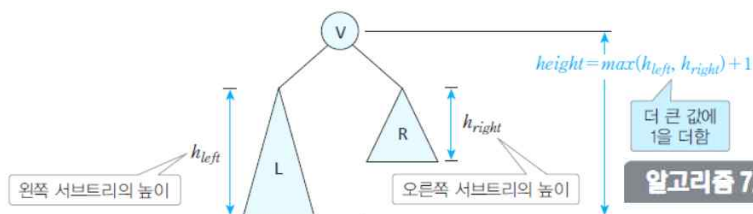
```

01 count_node(n)
02   if n = NULL :
03       return 0
04   n_left ← count_node(n.left)
05   n_right ← count_node(n.right)
06   return 1 + n_left + n_right

```

// 왼쪽 서브 트리 처리
// 오른쪽 서브 트리 처리
// 루트 노드 처리

트리 높이 구하기 : 각각 좌우 서브 트리에 대해 순환호출 한 후 max값 반환 +1



알고리즘 7.7

이진 트리의 높이 구하기

```

01 calc_height(n)
02   if n = NULL :
03       return 0
04   else : return 1 + max(calc_height(n.left), calc_height(n.right))

```

```
// 이진 트리의 노드 수 구하기
int count_node(TNode* n)
{
    if (n == NULL)
        return 0;
    else return 1 + count_node(n->left) + count_node(n->right);
}
```

```
#define MAX(a,b) ((a)>(b) ? (a):(b))

// 이진 트리의 높이 구하기
int calc_height(TNode* n)
{
    if (n == NULL)
        return 0;
    else return 1 + MAX(calc_height(n->left), calc_height(n->right));
}
```

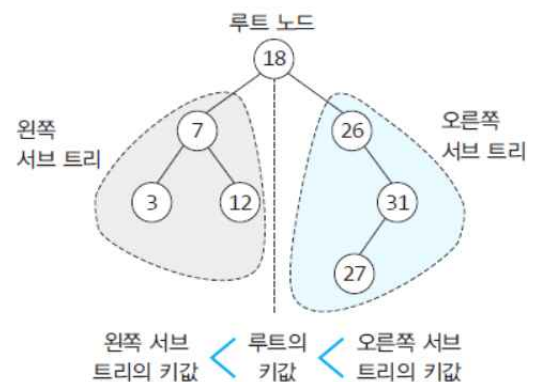
이진 탐색 트리 Binary Search Tree

탐색을 효율적으로 하기 위한 구조

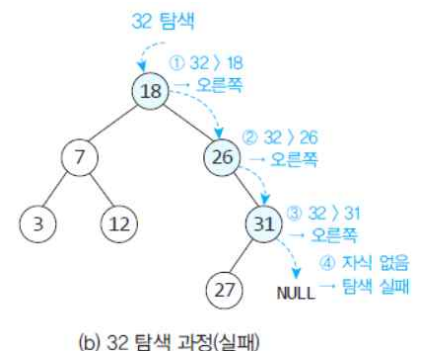
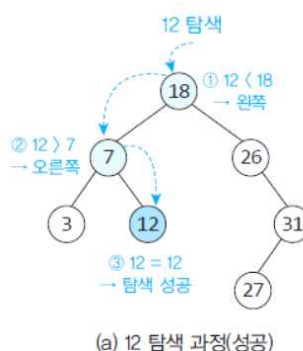
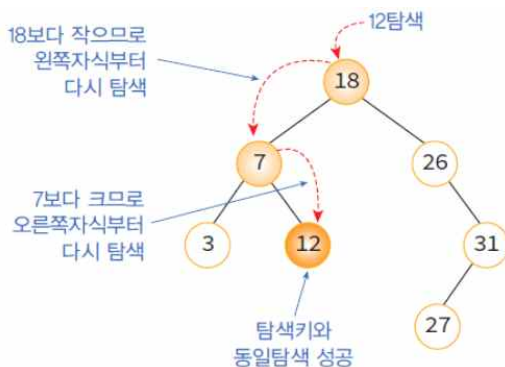
규칙 : 왼쪽 서브 트리 키값 < 루트 키값 < 오른쪽 서브 트리 키값

BST를 inorder 순회하면 오름차순 정렬이 됨

정의 : 모든 노드는 **유일한 키**를 갖는다 - 중복X
왼쪽 서브 트리 키는 루트의 키보다 **작다**
오른쪽 서브 트리의 키는 루트의 키보다 **크다**
 왼쪽과 오른쪽 서브 트리도 **이진 탐색 트리**이다



알고리즘 : 주어진 키 값이 루트 노드의 키 값보다 **작으면** -> 루트 노드의 왼쪽 자식을 기준으로 재탐색
 주어진 키 값이 루트 노드의 키 값보다 **크면** -> 루트 노드의 오른쪽 자식을 기준으로 재탐색
 키 값과 루트 노드의 키 값이 **같으면** -> 탐색 성공



알고리즘 7.10 이진 탐색 트리의 탐색 알고리즘

```
01 search(root, key)
02     if root = NULL :
03         return NULL
04     if KEY(root) = key :           // 루트의 키가 key와 같으면 탐색 성공
05         return root
06     else if KEY(root) > key :       // 루트의 키가 key보다 크면
07         return search(root.left, key) // 왼쪽 서브 트리 탐색
08     else :                         // 루트의 키가 key보다 작으면
09         return search(root.right, key) // 오른쪽 서브 트리 탐색
```

순환과 반복 두 가지로 구현 가능!

```

TNode *search(TNode *node, int key)    //순환형태로 구현
{
    if ( node == NULL ) return NULL;
    if ( key == node->key ) return node;    //탐색 성공
    else if ( key < node->key )             //키가 노드의 키값보다 작으면
        return search(node->left, key);    //왼쪽 서브트리 검색
    else                                   //키가 노드의 키값보다 크면
        return search(node->right, key);   //오른쪽 서브트리 검색
}

```

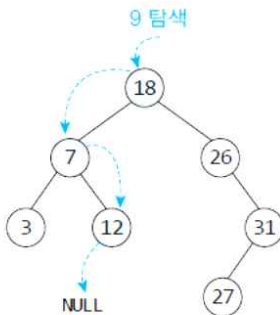
```

TNode *search(TNode *node, int key)    //반복형태로 구현
{
    while(node != NULL){
        if( key == node->key ) return node;
        else if( key < node->key )
            node = node->left;
        else
            node = node->right;
    }
    return NULL;    // 탐색에 실패시 NULL 반환
}

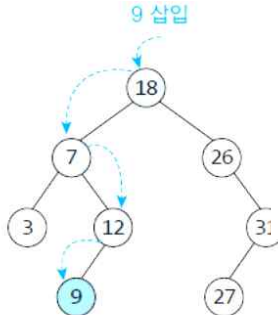
```

BST 삽입

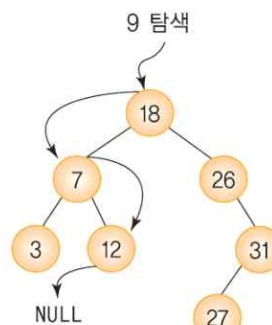
BST에 원소를 삽입하기 위해서는 **먼저 탐색** 수행
탐색에 **실패한 위치**가 새로운 노드 **삽입 위치**



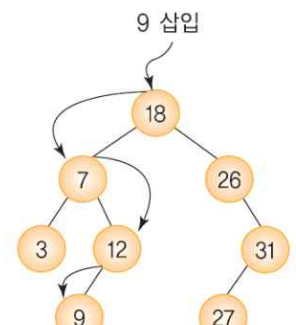
(a) 탐색을 먼저 수행



(b) 탐색이 실패한 위치에 노드 삽입



(a)



(b)

알고리즘 7.11 이진 탐색 트리의 삽입 알고리즘

```

01 insert(root, n)
02     if KEY(n) < KEY(root) :           // root보다 키가 작으면→ 왼쪽
03         if root.left = NULL :         // 왼쪽 자식이 없으면
04             root.left ← n             // n이 왼쪽 자식
05     else insert(root.left, n)          // 있으면 왼쪽 서브 트리에 삽입
06     else if KEY(n) > KEY(root) :       // root보다 키가 크면→ 오른쪽
07         if root.right = NULL :         // 오른쪽 자식이 없으면
08             root.right ← n            // n이 오른쪽 자식
09     else : insert(root.right, n)       // 있으면 오른쪽 서브 트리에 삽입
10     else :                             // 중복된 키가 있음.
11         delete node(n)                // 노드 n 삭제

```

>> 삭제는 할 필요 없음... 원래 중복허용X

```

TNode * insert_node(TNode * node, int key)
{
    // 트리가 공백이면 새로운 노드를 반환한다.
    if ( node == NULL ) return new_node(key);

    // 그렇지 않으면 순환적으로 트리를 내려간다.
    if ( key < node->key )
        node->left = insert_node(node->left, key);
    else if ( key > node->key )
        node->right = insert_node(node->right, key);

    // 변경된 루트 포인터를 반환한다.
    return node;
}

```

```

TNode * new_node(int item)
{
    TNode * temp = (TNode *)malloc(sizeof(TNode));
    temp->key = item;
    temp->left = temp->right = NULL;
    return temp;
}

```

BST 삭제

이진 탐색 트리 BST의 삭제 연산

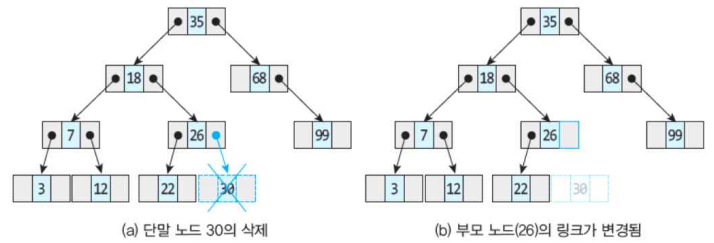
case 1. 단말 노드의 삭제

case 2. 자식이 하나인 노드의 삭제

case 3. 자식이 둘인 노드의 삭제

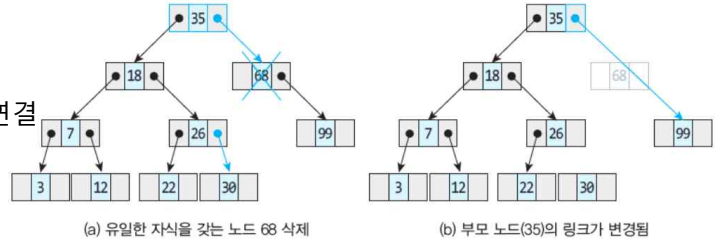
case 1. 단말 노드(leaf node)의 삭제

단말 노드의 부모 노드를 찾아서 연결을 끊으면 됨



case 2. 자식이 하나인 노드의 삭제

해당 노드를 삭제하고 서브 트리는 부모 노드에 연결



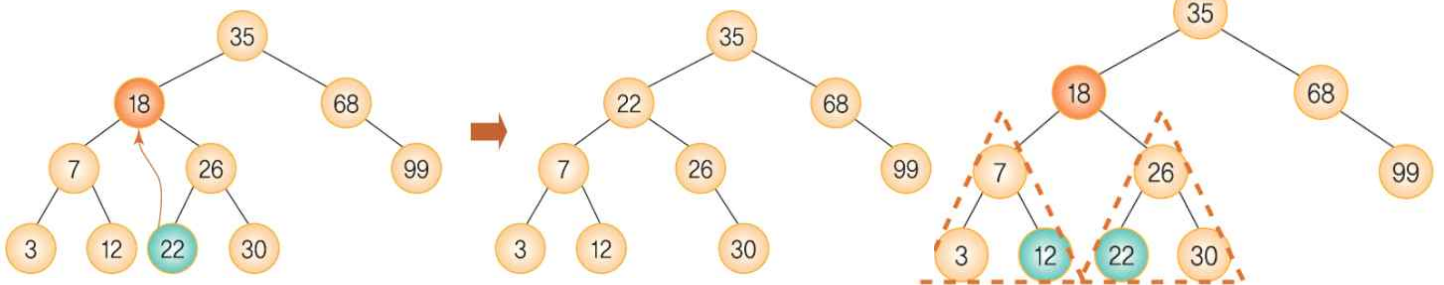
case 3. 자식이 둘인 노드의 삭제

삭제 노드와 가장 비슷한 값을 가진 노드(후계자 노드)를 삭제한 노드 위치로 가져옴.

그 후 후계자 노드는 삭제

후계자 노드 선정 : 삭제할 노드의 왼쪽 서브 트리에서 제일 큰 값이나,

오른쪽 서브트리에서 제일 작은 값 중에서 선택 (어느것이든 상관X)



```

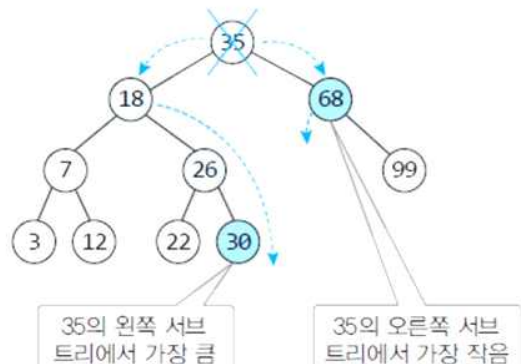
01 delete(root, key)
02     n ← 삭제할 노드
03     parent ← n의 부모 노드
04     if n = NULL : return root
05
06     // case1: n이 단말 노드인 경우
07     if n.left=NULL and n.right=NULL :
08         if parent=NULL : root ← NULL
09         else if parent.left = n :
10             parent.left ← NULL
11         else : parent.right ← NULL
12
13     // case1: n이 하나의 자식만 갖는 경우
14     else if n.left=NULL or n.right = NULL :
15         child ← n의 유일한 자식
16         if parent = NULL : root ← child
17         else if parent.left = n :
18             parent.left ← child
19         else : parent.right ← child
20

```

```

21     // case1: n이 양쪽 자식을 모두 갖는 경우
22     else :
23         succ ← n.right           // 오른쪽
24         while succ.left ≠ NULL :
25             succ ← succ.left
26         n.data ← succ.data       // 노드의
27         n.right ← delete(n.right, KEY(succ))
28
29     return root

```



```
TNode* delete_bst(TNode* root, int key)
```

```
{
    TNode* n = root;          // 삭제할 노드
    TNode* parent = NULL;     // 삭제할 노드의 부모
    while (n != NULL && key != n->data) {
        parent = n;
        n = (key < n->data) ? n->left : n->right;
    }
    if (n == NULL) return root; // 삭제할 노드가 없음

    // case 1: n이 단말 노드인 경우
    if ((n->left == NULL && n->right == NULL)) {
        if (parent == NULL) root = NULL;
        else {
            if (parent->left == n) parent->left = NULL;
            else parent->right = NULL;
        }
        free(n);
    }
    // case 2: n이 하나의 자식만 갖는 경우
    else if (n->left == NULL || n->right == NULL) {
        TNode* child = (n->left != NULL) ? n->left : n->right;
        if (n == root) root = child;
        else {
            if (parent->left == n) parent->left = child;
            else parent->right = child;
        }
        free(n);
    }
    // case 3: n이 양쪽 자식을 모두 갖는 경우
    else {
        TNode* succ = n->right;
        while (succ->left != NULL)
            succ = succ->left;
        n->data = succ->data;
        n->right = delete_bst(n->right, succ->data);
    }
    return root;
}
```

← 삭제할 노드와 그 부모 노드를 탐색

← 오른쪽 서브트리의 최소값 노드를 후계자로 사용하기 위해 탐색

← 오른쪽 서브트리에서 후계자를 찾아 삭제

BST 성능 분석

시간 복잡도 : $O(h)$ - 트리의 높이에 비례($h = \lceil \log \quad \rceil$)

$\therefore O(\log_2 n)$

최악인 경우 : 한쪽으로 치우친 경사이진트리의 경우. 순차탐색과 시간 복잡도가 동일. $O(n)$: $h=n$

Lab : 이진 트리를 좌우 대칭시키기!



기본 알고리즘

루트 노드를 기준으로 좌/우 서브 트리의 위치를 교환

각 서브 트리들도 개칭이 되도록 좌/우 교환 -> recursion

>> Preorder Traversal(전위 순회)를 사용해야 한다!

□ 알고리즘 → Preorder traversal (전위순회)

```
reverse_btree(root)
    root.left ↔ root.right    //루트 노드 좌우 서브트리 교환
    reverse_btree(root.left)  //루트 노드 왼쪽 서브트리 처리
    reverse_btree(root.right) //루트 노드 오른쪽 서브트리 처리
```

□ 구현

```
void reverse_btree(TNode* p)
{
    if (p != NULL) {
        TNode* tmp = p->left;
        p->left = p->right;
        p->right = tmp;

        reverse_btree(p->left);
        reverse_btree(p->right);
    }
}
```

9장 그래프 1

그래프(List)란?

연결되어 있는 객체 간의 관계를 표현하는 자료구조
 그래프의 예 : 지하철 노선도, 전기회로, 인맥지도, 지도, 등...

트리 : cycle을 허용하지 않는 그래프!

Traversal할때 나한테 돌아올 수 없음

그래프는 1800년대 오일러에 의해서 창안.

오일러의 문제 - 한붓그리기... 모든 노드에 연결된 간선 수가 짝수이면 가능

그래프 G는 (V, E)로 표시

정점 (Vertex) ~~ 노드

여러가지 특성을 가질 수 있는 객체

V(G): 그래프 G의 정점들의 집합

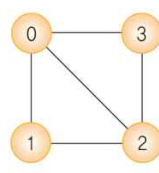
노드라고도 함

간선 (Edge) ~~ 링크

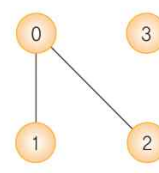
정점들 간의 관계

E(G): 그래프 G의 간선들의 집합

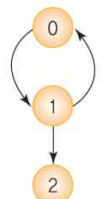
링크(link)라고도 함



G1



G2



G3

$V(G1) = \{0, 1, 2, 3\}$, $E(G1) = \{(0, 1), (0, 2), (0, 3), (1, 2), (2, 3)\}$

$V(G2) = \{0, 1, 2, 3\}$, $E(G2) = \{(0, 1), (0, 2)\}$

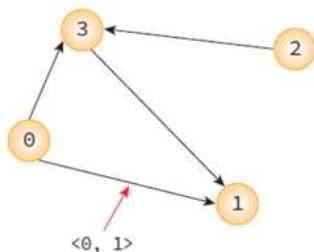
$V(G3) = \{0, 1, 2\}$, $E(G3) = \{<0, 1>, <1, 0>, <1, 2>\}$

Edge에서 () 이면 **무방향**

< > 이면 **방향**

무방향 그래프 : 간선을 통해서 양방향으로 갈 수 있음. ()로 표현. $(A, B) == (B, A)$

방향 그래프 : 간선을 통해서 한쪽 방향으로만 갈 수 있음(일방통행) $<A, B> != <B, A>$



$V(G2) = \{0, 1, 2, 3\}$,
 $E(G2) = \{<0, 1>, <0, 3>, <3, 1>, <2, 3>\}$

가중치 그래프 Weighted graph

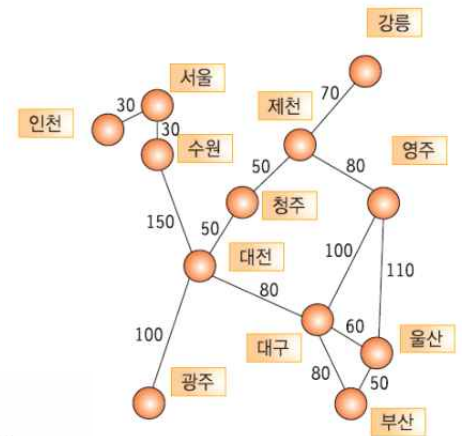
네트워크(Network) 라고도 함.

간선에 비용이나 가중치가 할당된 그래프

정점 : 각 도시

간선 : 도시를 연결하는 도로

가중치 : 도로의 길이

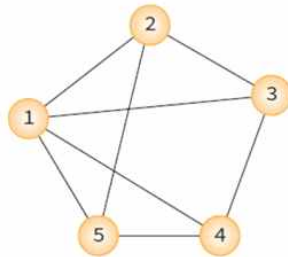


부분 그래프 (subgraph)

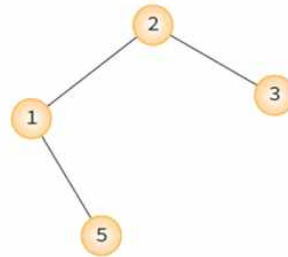
정점 집합 $V(G)$ 와 간선 집합 $E(G)$ 의 부분 집합으로 이루어진 그래프

$V(S) \subseteq V(G)$

$E(S) \subseteq E(G)$



(a) 그래프

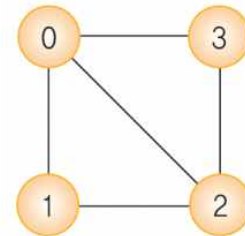


(b) 부분 그래프

인접 정점(adjacent vertex)

하나의 정점에서 간선에 의해 직접 연결된 정점.

ex) G_1 에서 정점 0의 인접 정점. - 정점1, 정점2, 정점3



차수(degree)

하나의 정점에 연결된 다른 정점의 수

무방향 그래프 : 차수의 합은 간선 수의 2배

ex) G_1 에서 정점 0의 차수 : 3

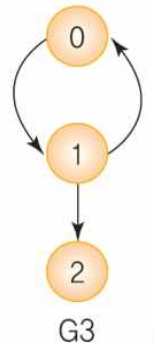
방향 그래프 :

진입차수 : 외부에서 오는 간선의 수

진출차수 : 외부로 향하는 간선의 수

방향 그래프의 모든 진입(or 진출) 차수의 합은 간선의 수

ex) G_3 의 진입차수의 합 : 3



경로(path)

무방향 그래프 : 정점 s 로부터 정점 e 까지의 경로

정점의 나열 : $s, v_1, v_2, \dots, v_k, e$

정점들간 반드시 간선 $(s, v_1), (v_1, v_2), \dots, (v_k, e)$ 가 존재해야 함

방향 그래프 : 정점 s 로부터 정점 e 까지의 경로

정점의 나열 : $s, v_1, v_2, \dots, v_k, e$

정점들간 반드시 간선 $\langle s, v_1 \rangle, \langle v_1, v_2 \rangle, \dots, \langle v_k, e \rangle$ 가 존재해야 함

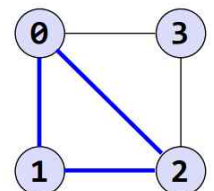
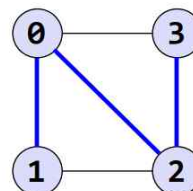
경로의 길이(length)

경로를 구성하는데 사용된 간선의 수

단순 경로(simple path)

경로 중에서 반복되는 간선이 없는 경로

ex) 1,0,2,3은 단순경로. 1,0,2,0은 단순경로 아님



사이클(cycle)

단순 경로의 시작 정점과 종료 정점이 동일한 경로

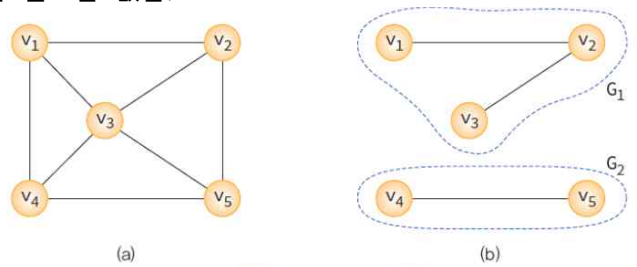
ex) 0,1,2,0은 사이클.

연결 그래프(connected graph)

모든 정점쌍에 대한 경로가 존재. 꼭 직접 연결되어야 할 필요는 없음.

한 노드에서 다른 어떤 노드로도 이동할 수 있어야 함.

ex) a는 연결 그래프. b는 비연결 그래프



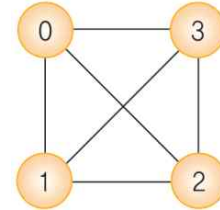
완전 그래프(complete graph)

모든 정점이 연결되어 있는 그래프

n개의 정점을 가진 무방향 완전그래프의 간선의 수: $n \times (n-1)$

n개 중에 2개를 뽑는 것과 같음.

ex) $n=4$ $(4 \times 3)/2=6$



트리(tree)

그래프의 특수한 형태. 사이클을 가지지 않는 연결 그래프

그래프의 ADT

데이터: 정점의 집합과 간선의 집합

연산

- `is_empty()`: 그래프가 공백 상태인지를 검사한다.
- `insert_vertex(v)`: 그래프에 정점 `v`를 삽입한다.
- `insert_edge(u,v)`: 그래프에 간선 `(u,v)`를 삽입한다.
- `delete_vertex(v)`: 정점 `v`를 그래프에서 삭제한다.
- `delete_edge(u,v)`: 간선 `(u,v)`를 그래프에서 삭제한다.
- `adjacent(v)`: 정점 `v`의 모든 인접 정점을 반환한다.
- `degree(v)`: 정점 `v`의 차수를 반환한다.

인접 행렬

방법.

간선 `(i,j)`가 그래프에 존재하면 ----- $M[i][j] = 1$
존재하지 않으면 ---- $M[i][j] = 0$

무방향 그래프는 항상 대칭행렬이다

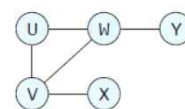
방향 그래프는 대칭이 아니다.

```
03 #define MAX_VSIZE 100 // 최대 정점 수
04 typedef char VElement; // VElement의 자료형 정의
05
06 int vsize = 5; // 그래프의 정점 수
07 VElement vdata[MAX_VSIZE] = { 'U', 'V', 'W', 'X', 'Y' };
08 int adj[MAX_VSIZE][MAX_VSIZE] = {
09     { 0, 1, 1, 0, 0 },
10     { 1, 0, 1, 1, 0 },
11     { 1, 1, 0, 0, 1 },
12     { 0, 1, 0, 0, 0 },
13     { 0, 0, 1, 0, 0 },
14 };
```

```
16 int degree(int v)
17 {
18     int count = 0;
19     for (int i = 0; i < vsize; i++)
20         if (adj[v][i] == 1) count++;
21     return count;
22 }
```

그림 8.5의 그래프에 대한
인접 행렬 표현

정점 v의 차수를 구하는 함수



(a) 무방향 그래프

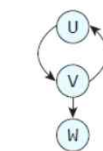
	0	1	2	3	4
U → 0	0	1	1	0	0
V → 1	1	0	1	1	0
W → 2	1	1	0	0	1
X → 3	0	1	0	0	0
Y → 4	0	0	1	0	0

간선 (U,Y)
는 없음

간선 (W,Y)
가 있음

무방향 그래프는
항상 대칭 행렬

(b) 간선의 인접 행렬 표현



(a) 방향 그래프

	0	1	2
U → 0	0	1	0
V → 1	1	0	1
W → 2	0	0	0

방향 그래프에서는
대칭 행렬이 아님

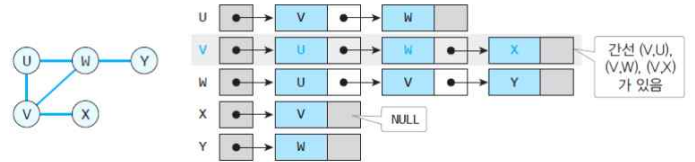
(b) 간선의 인접 행렬 표현

인접 리스트

각 정점에 **인접한 정점들**을 linked list로 표현

ex) V의 인접한 정점은 U W X이다

순서는 중요하지 않음!



```
#define MAX_VSIZE 100 // 최대 정점 수
typedef char VElement; // VElement의 자료형 정의

typedef struct GraphNode {
    int id; // 인접 정점의 id
    struct GraphNode* link; // 다음 노드의 포인터
} GNode;

int vsize = 5; // 정점의 수
VElement vdata[5] = { 'U', 'V', 'W', 'X', 'Y' };
GNode* adj[5] = { NULL, NULL, NULL, NULL, NULL };

// 새로운 노드를 리스트의 맨 앞에 추가 (효율성)
GNode* prepend_node(GNode* head, int v)
{
    GNode* n = (GNode*)malloc(sizeof(GNode));
    n->id = v;
    n->link = head;
    return n;
}
```

// 무방향 그래프에서는 (u,v) 간선 하나가 추가되면 연결 리스트에서는
// <u,v>와 <v,u> 두 개가 각각 리스트에 추가 되어야함

```
void insert_edge(int u, int v)
```

```
{
    adj[u] = prepend_node(adj[u], v);
    adj[v] = prepend_node(adj[v], u);
}
```

// 정점의 차수를 계산
int degree(int v)

```
{
    int count = 0;
    for (GNode* p = adj[v]; p != NULL; p = p->link)
        count++;
    return count;
}
```

append(뒤에 붙이기)가 아니라 prepend(앞에 붙이기)를 하는 이유

>> 어차피 순서가 없기 때문에 굳이 끝까지 따라가서 뒤에 붙일 이유가 없다

insert_edge에서 prepend를 2번 하는 이유는, (u,v)를 추가하려면 u입장에서 v를 추가하기도 해야하고, v입장에서 u를 추가하기도 해야해서 2개의 노드를 만들어야 한다.

인접행렬

무조건 n^2 의 공간 필요
조밀 그래프에서 효과적

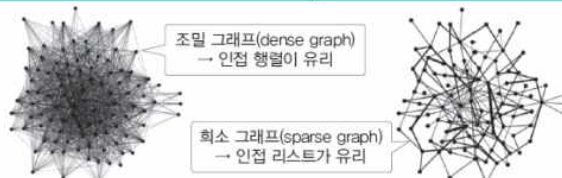
인접리스트

$n+2e$ 만큼 필요
희소 그래프에서 효과적

>> 일장일단이 있다!

인접행렬	인접리스트
간선의 수에 무관하게 항상 n^2 개의 메모리 공간이 필요하다. 따라서 정점에 비해 간선의 수가 매우 많은 <u>조밀 그래프(dense graph)</u> 에서 효과적이다.	n 개의 연결 리스트가 필요하고, $2e$ 개의 노드가 필요하다. 즉 $n+2e$ 개의 메모리 공간이 필요하다. 따라서 정점에 비해 간선의 개수가 매우 적은 <u>희소 그래프(sparse graph)</u> 에서 효과적이다.
u와 v를 연결하는 간선 (u,v)의 유무는 인접 행렬의 요소 $adj[u][v]$ 를 조사하면 바로 알 수 있다. 따라서 $O(1)$ 이다.	(u,v)가 있는지는 u의 연결 리스트 전체를 조사해야 알 수 있다. 정점 u의 차수를 d_u 라고 한다면 시간 복잡도는 $O(d_u)$ 이다.
v의 차수는 인접 행렬에서 v에 해당하는 행의 모든 성분을 조사해야 알 수 있다. 따라서 복잡도는 $O(n)$ 이다.	v의 차수는 v의 연결 리스트의 길이이다. 따라서 복잡도는 v의 차수에 비례하는 $O(d_v)$ 이다.
v의 모든 인접 정점을 구하려면 해당 행의 모든 요소를 조사해야 하므로 $O(n)$ 의 시간이 요구된다.	정점 v의 연결 리스트의 모든 요소를 방문해야 하므로 $O(d_v)$ 이다.
그래프의 전체 간선 수를 구하려면 인접 행렬 전체를 조사해야 하므로 $O(n^2)$ 의 시간이 요구된다.	모든 인접 리스트의 모든 요소를 방문해야 하므로 $O(n+e)$ 의 연산이 요구된다.

양태학교
YOUNG TAE HAN



그래프 탐색

하나의 정점에서 차례대로 **모든 정점들**을 한번씩 방문

그래프 탐색 응용

많은 문제들이 단순히 그래프 노드를 **탐색하는 것만으로도 해결되는 경우가 많음**

ex) 도로망에서 특정도시에서 다른 도시로 갈 수 있는지 여부

ex) 전자회로에서 단자들 연결성 검사

ex) 미로 탐색 문제